



SIMATS ENGINEERING

CO3 ASSESSMENT TOOL 2

NAME:K.HARSHA VARDHAN-192425228

1. Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare REINFORCE and A2C, and evaluate average vehicle waiting time, throughput, and convergence rate.

In [7]:

```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

torch.manual_seed(42)
random.seed(42)
np.random.seed(42)

EPISODES = 300
STEPS = 50
GAMMA = 0.95
LEARNING_RATE = 0.001

class TrafficEnvironment:

    def __init__(self):
        self.max_cars = 20
        self.reset()

    def reset(self):
        self.north = random.randint(5, 15)
        self.south = random.randint(5, 15)
        self.east = random.randint(5, 15)
        self.west = random.randint(5, 15)

        self.signal = 0

        return self.get_state()

    def get_state(self):

        return np.array([
            self.north / self.max_cars,
            self.south / self.max_cars,
            self.east / self.max_cars,
            self.west / self.max_cars,
            self.signal
        ], dtype=np.float32)

    def step(self, action):

        self.signal = action

        if action == 0:

            passed_north = min(self.north, 5)
            passed_south = min(self.south, 5)
```

```
        self.north -= passed_north
        self.south -= passed_south

        throughput = passed_north + passed_south

    else:

        passed_east = min(self.east, 5)
        passed_west = min(self.west, 5)

        self.east -= passed_east
        self.west -= passed_west

        throughput = passed_east + passed_west

    new_north = random.randint(0, 3)
    new_south = random.randint(0, 3)
    new_east = random.randint(0, 3)
    new_west = random.randint(0, 3)

    self.north = min(self.north + new_north, self.max_cars)
    self.south = min(self.south + new_south, self.max_cars)
    self.east = min(self.east + new_east, self.max_cars)
    self.west = min(self.west + new_west, self.max_cars)

    waiting = (
        self.north +
        self.south +
        self.east +
        self.west
    )

    reward = throughput - 0.1 * waiting

    next_state = self.get_state()

    return next_state, reward, waiting, throughput

class PolicyNetwork(nn.Module):

    def __init__(self):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(5, 32),
            nn.ReLU(),
            nn.Linear(32, 32),
            nn.ReLU(),
            nn.Linear(32, 2),
            nn.Softmax(dim=-1)
        )
```

```
def forward(self, state):
    return self.network(state)

class A2CNetwork(nn.Module):

    def __init__(self):
        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(5, 32),
            nn.ReLU(),
            nn.Linear(32, 32),
            nn.ReLU()
        )

        self.actor = nn.Linear(32, 2)

        self.critic = nn.Linear(32, 1)

    def forward(self, state):

        x = self.shared(state)

        action_prob = torch.softmax(
            self.actor(x),
            dim=-1
        )

        value = self.critic(x)

        return action_prob, value

def train_reinforce():

    env = TrafficEnvironment()

    model = PolicyNetwork()

    optimizer = optim.Adam(
        model.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []
    waiting_history = []
    throughput_history = []

    for episode in range(EPISODES):

        state = env.reset()
```

```
log_probs = []
rewards = []

total_waiting = 0
total_throughput = 0

for step in range(STEPS):

    state_tensor = torch.FloatTensor(state)

    probabilities = model(state_tensor)

    distribution = torch.distributions.Categorical(
        probabilities
    )

    action = distribution.sample()

    log_prob = distribution.log_prob(action)

    next_state, reward, waiting, throughput = env.step(
        action.item()
    )

    log_probs.append(log_prob)
    rewards.append(reward)

    total_waiting += waiting
    total_throughput += throughput

    state = next_state

returns = []

G = 0

for reward in reversed(rewards):

    G = reward + GAMMA * G

    returns.insert(0, G)

returns = torch.tensor(
    returns,
    dtype=torch.float32
)

if len(returns) > 1:

    returns = (
        returns - returns.mean()
    ) / (returns.std() + 1e-8)
```

```
loss = 0

for log_prob, G in zip(
    log_probs,
    returns
):

    loss += -log_prob * G

optimizer.zero_grad()

loss.backward()

optimizer.step()

reward_history.append(sum(rewards))

waiting_history.append(
    total_waiting / STEPS
)

throughput_history.append(
    total_throughput
)

return (
    model,
    reward_history,
    waiting_history,
    throughput_history
)

def train_a2c():

    env = TrafficEnvironment()

    model = A2CNetwork()

    optimizer = optim.Adam(
        model.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []
    waiting_history = []
    throughput_history = []

    for episode in range(EPISODES):

        state = env.reset()

        log_probs = []
```

```
values = []
rewards = []

total_waiting = 0
total_throughput = 0

for step in range(STEPS):

    state_tensor = torch.FloatTensor(state)

    probabilities, value = model(
        state_tensor
    )

    distribution = torch.distributions.Categorical(
        probabilities
    )

    action = distribution.sample()

    log_prob = distribution.log_prob(action)

    next_state, reward, waiting, throughput = env.step(
        action.item()
    )

    log_probs.append(log_prob)

    values.append(value.squeeze())

    rewards.append(reward)

    total_waiting += waiting
    total_throughput += throughput

    state = next_state

returns = []

G = 0

for reward in reversed(rewards):

    G = reward + GAMMA * G

    returns.insert(0, G)

returns = torch.tensor(
    returns,
    dtype=torch.float32
)

values = torch.stack(values)
```

```
advantages = returns - values.detach()

actor_loss = torch.tensor(0.0)

for log_prob, advantage in zip(
    log_probs,
    advantages
):

    actor_loss += -log_prob * advantage

critic_loss = nn.functional.mse_loss(
    values,
    returns
)

loss = actor_loss + 0.5 * critic_loss

optimizer.zero_grad()

loss.backward()

optimizer.step()

reward_history.append(sum(rewards))

waiting_history.append(
    total_waiting / STEPS
)

throughput_history.append(
    total_throughput
)

return (
    model,
    reward_history,
    waiting_history,
    throughput_history
)

def calculate_convergence(rewards):

    final_average = np.mean(
        rewards[-30:]
    )

    target = 0.90 * final_average

    window = 20
```

```

    for i in range(
        window,
        len(rewards)
    ):

        average = np.mean(
            rewards[i-window:i]
        )

        if average >= target:

            return i + 1

    return len(rewards)

print("=====")
print("INTELLIGENT TRAFFIC SIGNAL CONTROL")
print("REINFORCE vs A2C")
print("=====")

print("\nTraining REINFORCE...")

reinforce_model, reinforce_rewards, reinforce_waiting, reinforce_throughput
= train_reinforce()

print("REINFORCE Training Completed")

print("\nTraining A2C...")

a2c_model, a2c_rewards, a2c_waiting, a2c_throughput = train_a2c()

print("A2C Training Completed")

reinforce_avg_reward = np.mean(
    reinforce_rewards[-30:]
)

a2c_avg_reward = np.mean(
    a2c_rewards[-30:]
)

reinforce_avg_waiting = np.mean(
    reinforce_waiting[-30:]
)

a2c_avg_waiting = np.mean(
    a2c_waiting[-30:]
)

reinforce_avg_throughput = np.mean(
    reinforce_throughput[-30:]
)

```

```

)

a2c_avg_throughput = np.mean(
    a2c_throughput[-30:]
)

reinforce_convergence = calculate_convergence(
    reinforce_rewards
)

a2c_convergence = calculate_convergence(
    a2c_rewards
)

print("\n=====")
print("FINAL RESULTS")
print("=====")

print("\nREINFORCE")
print("Average Reward      :", round(reinforce_avg_reward, 2))
print("Average Waiting Time :", round(reinforce_avg_waiting, 2))
print("Average Throughput    :", round(reinforce_avg_throughput, 2))
print("Convergence Episode   :", reinforce_convergence)

print("\nA2C")
print("Average Reward      :", round(a2c_avg_reward, 2))
print("Average Waiting Time :", round(a2c_avg_waiting, 2))
print("Average Throughput    :", round(a2c_avg_throughput, 2))
print("Convergence Episode   :", a2c_convergence)

print("\n=====")
print("COMPARISON")
print("=====")

print(
    "\nMetric".ljust(25),
    "REINFORCE".ljust(15),
    "A2C"
)

print(
    "Average Waiting Time".ljust(25),
    round(reinforce_avg_waiting, 2),
    " " * 8,
    round(a2c_avg_waiting, 2)
)

print(
    "Average Throughput".ljust(25),
    round(reinforce_avg_throughput, 2),
    " " * 8,

```

```
        round(a2c_avg_throughput, 2)
    )

    print(
        "Convergence Episode".ljust(25),
        reinforce_convergence,
        " " * 8,
        a2c_convergence
    )

    if a2c_avg_waiting < reinforce_avg_waiting:
        print("\nA2C has lower average waiting time.")

    else:
        print("\nREINFORCE has lower average waiting time.")

    if a2c_avg_throughput > reinforce_avg_throughput:
        print("A2C has higher throughput.")

    else:
        print("REINFORCE has higher throughput.")

    if a2c_convergence < reinforce_convergence:
        print("A2C converges faster.")

    else:
        print("REINFORCE converges faster.")

    plt.figure(figsize=(10, 5))

    plt.plot(
        reinforce_rewards,
        label="REINFORCE"
    )

    plt.plot(
        a2c_rewards,
        label="A2C"
    )

    plt.xlabel("Episode")
    plt.ylabel("Total Reward")
    plt.title("Reward Comparison")
    plt.legend()
    plt.grid()

    plt.show()
```

```

plt.figure(figsize=(10, 5))

plt.plot(
    reinforce_waiting,
    label="REINFORCE"
)

plt.plot(
    a2c_waiting,
    label="A2C"
)

plt.xlabel("Episode")
plt.ylabel("Average Waiting Time")
plt.title("Vehicle Waiting Time Comparison")
plt.legend()
plt.grid()

plt.show()

plt.figure(figsize=(10, 5))

plt.plot(
    reinforce_throughput,
    label="REINFORCE"
)

plt.plot(
    a2c_throughput,
    label="A2C"
)

plt.xlabel("Episode")
plt.ylabel("Vehicles Passed")
plt.title("Traffic Throughput Comparison")
plt.legend()
plt.grid()

plt.show()

print("\nProject Completed Successfully.")

```

```

=====
INTELLIGENT TRAFFIC SIGNAL CONTROL
REINFORCE vs A2C
=====

```

```

Training REINFORCE...
REINFORCE Training Completed

```

```

Training A2C...

```

A2C Training Completed

=====

FINAL RESULTS

=====

REINFORCE

Average Reward : 250.57

Average Waiting Time : 15.38

Average Throughput : 327.47

Convergence Episode : 21

A2C

Average Reward : 236.12

Average Waiting Time : 17.84

Average Throughput : 325.33

Convergence Episode : 21

=====

COMPARISON

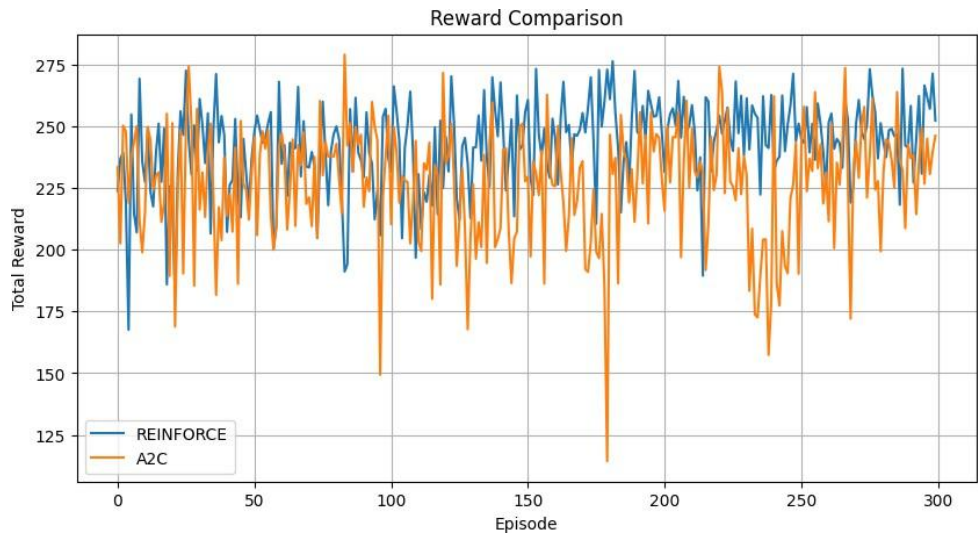
=====

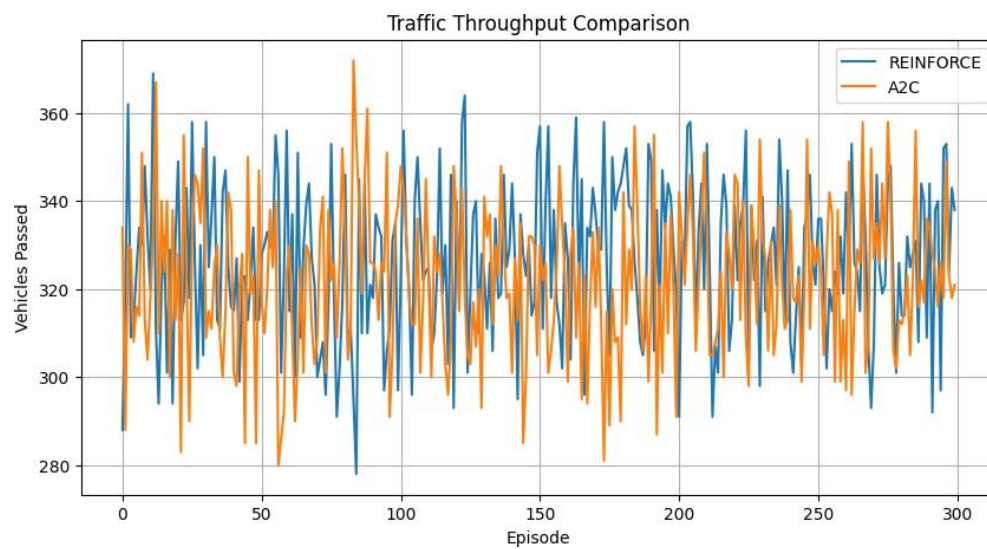
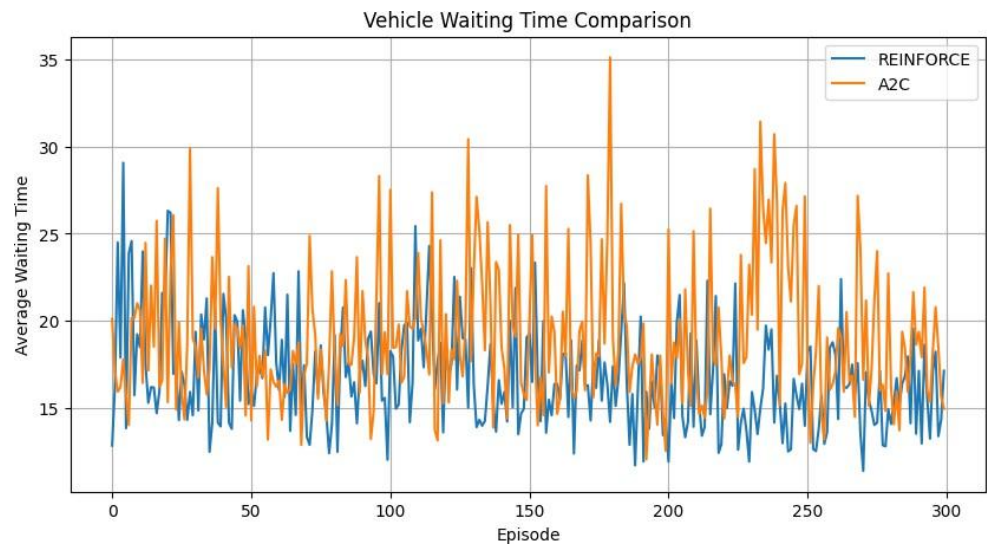
Metric	REINFORCE	A2C
Average Waiting Time	15.38	17.84
Average Throughput	327.47	325.33
Convergence Episode	21	21

REINFORCE has lower average waiting time.

REINFORCE has higher throughput.

REINFORCE converges faster.





Project Completed Successfully.

2. Design and implement an Actor-Critic (A2C/A3C) model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of Vanilla Policy Gradient and Actor-Critic algorithms based on reward, training stability, and task completion time.

```
In [8]: import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

EPISODES = 300
MAX_STEPS = 60
GRID_SIZE = 5
GAMMA = 0.95
LEARNING_RATE = 0.001

class WarehouseEnvironment:

    def __init__(self):
        self.size = GRID_SIZE
        self.reset()

    def reset(self):

        self.robot = [0, 0]
        self.package = [4, 4]
        self.goal = [0, 4]

        self.obstacles = [
            [1, 1],
            [1, 2],
            [2, 2],
            [3, 2]
        ]

        self.delivered = False

        return self.get_state()

    def get_state(self):

        state = [
            self.robot[0] / 4,
            self.robot[1] / 4,
            self.package[0] / 4,
            self.package[1] / 4,
            self.goal[0] / 4,
            self.goal[1] / 4
        ]

        return np.array(state, dtype=np.float32)
```

```
def step(self, action):

    old_position = self.robot.copy()

    if action == 0:
        self.robot[0] -= 1

    elif action == 1:
        self.robot[0] += 1

    elif action == 2:
        self.robot[1] -= 1

    elif action == 3:
        self.robot[1] += 1

    reward = -1
    done = False

    if (
        self.robot[0] < 0 or
        self.robot[0] >= self.size or
        self.robot[1] < 0 or
        self.robot[1] >= self.size
    ):

        self.robot = old_position
        reward = -3

    elif self.robot in self.obstacles:

        self.robot = old_position
        reward = -5

    elif self.robot == self.package and not self.delivered:

        reward = 10
        self.delivered = True

    elif self.delivered and self.robot == self.goal:

        reward = 30
        done = True

    next_state = self.get_state()

    return next_state, reward, done


class PolicyNetwork(nn.Module):

    def __init__(self):
```

```
super().__init__()

self.network = nn.Sequential(
    nn.Linear(6, 64),
    nn.ReLU(),
    nn.Linear(64, 64),
    nn.ReLU(),
    nn.Linear(64, 4),
    nn.Softmax(dim=-1)
)

def forward(self, state):

    return self.network(state)

class A2CNetwork(nn.Module):

    def __init__(self):

        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(6, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU()
        )

        self.actor = nn.Linear(64, 4)
        self.critic = nn.Linear(64, 1)

    def forward(self, state):

        x = self.shared(state)

        action_probability = torch.softmax(
            self.actor(x),
            dim=-1
        )

        value = self.critic(x)

        return action_probability, value

def train_reinforce():

    env = WarehouseEnvironment()

    model = PolicyNetwork()
```

```
optimizer = optim.Adam(
    model.parameters(),
    lr=LEARNING_RATE
)

rewards_history = []
steps_history = []
completed_history = []

for episode in range(EPIISODES):

    state = env.reset()

    log_probs = []
    rewards = []

    completed = 0

    for step in range(MAX_STEPS):

        state_tensor = torch.tensor(
            state,
            dtype=torch.float32
        )

        probabilities = model(state_tensor)

        distribution = torch.distributions.Categorical(
            probabilities
        )

        action = distribution.sample()

        log_prob = distribution.log_prob(action)

        next_state, reward, done = env.step(
            action.item()
        )

        log_probs.append(log_prob)
        rewards.append(reward)

        state = next_state

        if done:

            completed = 1
            break

    returns = []

    G = 0
```

```
for reward in reversed(rewards):

    G = reward + GAMMA * G

    returns.insert(0, G)

returns = torch.tensor(
    returns,
    dtype=torch.float32
)

if len(returns) > 1:

    returns = (
        returns - returns.mean()
    ) / (returns.std() + 1e-8)

loss = 0

for log_prob, G in zip(
    log_probs,
    returns
):

    loss += -log_prob * G

optimizer.zero_grad()

loss.backward()

optimizer.step()

rewards_history.append(
    sum(rewards)
)

steps_history.append(
    step + 1
)

completed_history.append(
    completed
)

return (
    model,
    rewards_history,
    steps_history,
    completed_history
)

def train_a2c():
```

```
env = WarehouseEnvironment()

model = A2CNetwork()

optimizer = optim.Adam(
    model.parameters(),
    lr=LEARNING_RATE
)

rewards_history = []
steps_history = []
completed_history = []

for episode in range(EPISODES):

    state = env.reset()

    log_probs = []
    values = []
    rewards = []

    completed = 0

    for step in range(MAX_STEPS):

        state_tensor = torch.tensor(
            state,
            dtype=torch.float32
        )

        probabilities, value = model(
            state_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        action = distribution.sample()

        log_prob = distribution.log_prob(action)

        next_state, reward, done = env.step(
            action.item()
        )

        log_probs.append(log_prob)
        values.append(value.squeeze())
        rewards.append(reward)

        state = next_state
```

```
        if done:

            completed = 1
            break

    returns = []

    G = 0

    for reward in reversed(rewards):

        G = reward + GAMMA * G

        returns.insert(0, G)

    returns = torch.tensor(
        returns,
        dtype=torch.float32
    )

    values = torch.stack(values)

    advantages = returns - values.detach()

    actor_loss = 0

    for log_prob, advantage in zip(
        log_probs,
        advantages
    ):

        actor_loss += -log_prob * advantage

    critic_loss = nn.functional.mse_loss(
        values,
        returns
    )

    loss = actor_loss + 0.5 * critic_loss

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    rewards_history.append(
        sum(rewards)
    )

    steps_history.append(
        step + 1
    )
```

```

        completed_history.append(
            completed
        )

    return (
        model,
        rewards_history,
        steps_history,
        completed_history
    )

print("=====")
print("WAREHOUSE ROBOT RL PROJECT")
print("VANILLA POLICY GRADIENT vs A2C")
print("=====")

print("\nTraining Vanilla Policy Gradient...")

(
    reinforce_model,
    reinforce_rewards,
    reinforce_steps,
    reinforce_completed
) = train_reinforce()

print("Vanilla Policy Gradient Training Completed.")

print("\nTraining A2C...")

(
    a2c_model,
    a2c_rewards,
    a2c_steps,
    a2c_completed
) = train_a2c()

print("A2C Training Completed.")

window = 30

reinforce_reward = np.mean(
    reinforce_rewards[-window:]
)

a2c_reward = np.mean(
    a2c_rewards[-window:]
)

reinforce_steps_avg = np.mean(
    reinforce_steps[-window:]
)

```

```

)

a2c_steps_avg = np.mean(
    a2c_steps[-window:]
)

reinforce_completion = (
    np.mean(reinforce_completed[-window:]) * 100
)

a2c_completion = (
    np.mean(a2c_completed[-window:]) * 100
)

print("\n=====")
print("FINAL PERFORMANCE")
print("=====")

print("\nVanilla Policy Gradient")
print("Average Reward      :", round(reinforce_reward, 2))
print("Average Steps       :", round(reinforce_steps_avg, 2))
print("Task Completion Rate :", round(reinforce_completion, 2), "%")

print("\nActor-Critic A2C")
print("Average Reward      :", round(a2c_reward, 2))
print("Average Steps       :", round(a2c_steps_avg, 2))
print("Task Completion Rate :", round(a2c_completion, 2), "%")

print("\n=====")
print("COMPARISON")
print("=====")

print(
    "\nMetric".ljust(25),
    "Policy Gradient".ljust(20),
    "A2C"
)

print(
    "Average Reward".ljust(25),
    round(reinforce_reward, 2),
    " " * 10,
    round(a2c_reward, 2)
)

print(
    "Average Steps".ljust(25),
    round(reinforce_steps_avg, 2),
    " " * 10,
    round(a2c_steps_avg, 2)
)

```

```
print(
    "Completion Rate (%).ljust(25),
    round(reinforce_completion, 2),
    " " * 10,
    round(a2c_completion, 2)
)

if a2c_reward > reinforce_reward:
    print("\nA2C achieved higher average reward.")

else:
    print("\nVanilla Policy Gradient achieved higher average reward.")

if a2c_steps_avg < reinforce_steps_avg:
    print("A2C completed tasks in fewer steps.")

else:
    print("Vanilla Policy Gradient completed tasks in fewer steps.")

if a2c_completion > reinforce_completion:
    print("A2C achieved better task completion.")

else:
    print("Vanilla Policy Gradient achieved better task completion.")

plt.figure(figsize=(10, 5))

plt.plot(
    reinforce_rewards,
    label="Vanilla Policy Gradient"
)

plt.plot(
    a2c_rewards,
    label="A2C"
)

plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Training Reward Comparison")
plt.legend()
plt.grid()

plt.show()

plt.figure(figsize=(10, 5))
```

```

plt.plot(
    reinforce_steps,
    label="Vanilla Policy Gradient"
)

plt.plot(
    a2c_steps,
    label="A2C"
)

plt.xlabel("Episode")
plt.ylabel("Steps")
plt.title("Task Completion Time Comparison")
plt.legend()
plt.grid()

plt.show()

plt.figure(figsize=(10, 5))

reinforce_running = np.cumsum(
    reinforce_completed
)

a2c_running = np.cumsum(
    a2c_completed
)

plt.plot(
    reinforce_running,
    label="Vanilla Policy Gradient"
)

plt.plot(
    a2c_running,
    label="A2C"
)

plt.xlabel("Episode")
plt.ylabel("Completed Tasks")
plt.title("Task Completion Comparison")
plt.legend()
plt.grid()

plt.show()

print("\n=====")
print("PROJECT COMPLETED")
print("=====")

```

=====

WAREHOUSE ROBOT RL PROJECT

VANILLA POLICY GRADIENT vs A2C

=====

Training Vanilla Policy Gradient...

Vanilla Policy Gradient Training Completed.

Training A2C...

A2C Training Completed.

=====

FINAL PERFORMANCE

=====

Vanilla Policy Gradient

Average Reward : -95.0

Average Steps : 56.1

Task Completion Rate : 30.0 %

Actor-Critic A2C

Average Reward : -109.57

Average Steps : 60.0

Task Completion Rate : 0.0 %

=====

COMPARISON

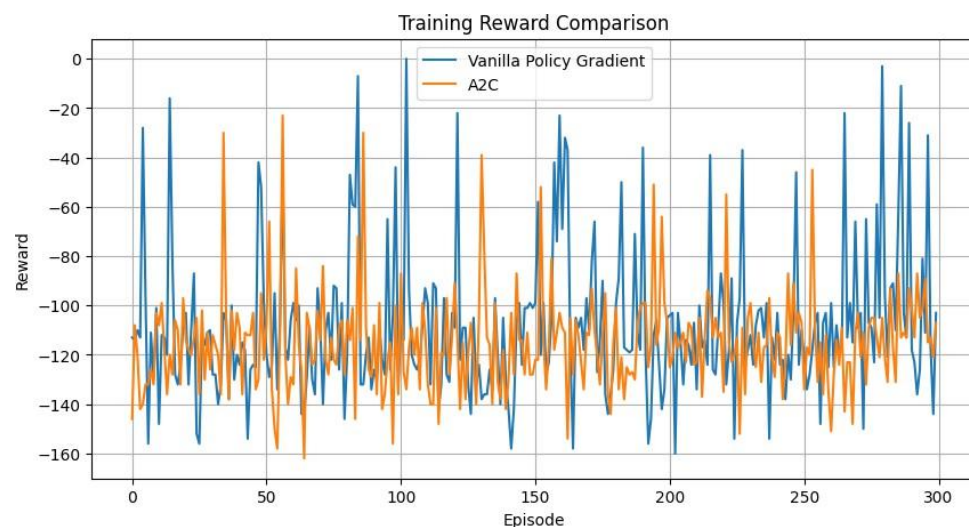
=====

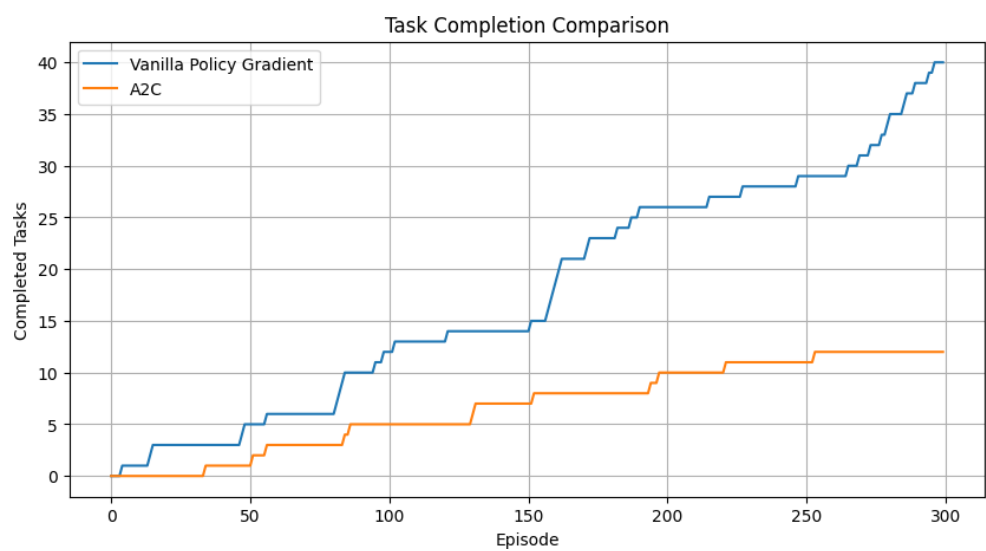
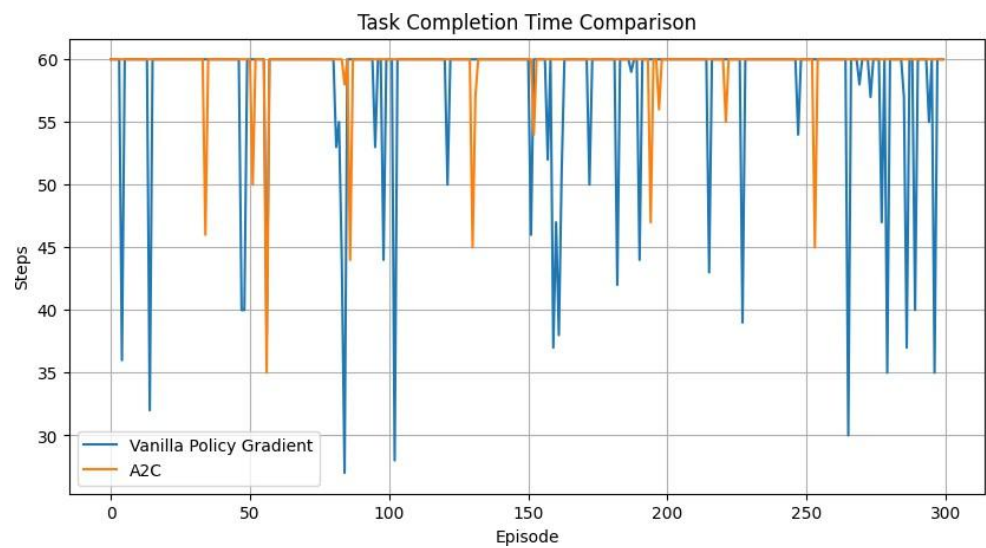
Metric	Policy Gradient	A2C
Average Reward	-95.0	-109.57
Average Steps	56.1	60.0
Completion Rate (%)	30.0	0.0

Vanilla Policy Gradient achieved higher average reward.

Vanilla Policy Gradient completed tasks in fewer steps.

Vanilla Policy Gradient achieved better task completion.





=====

PROJECT COMPLETED

=====

3. Develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

```
In [9]: import numpy as np
import random
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from collections import deque

np.random.seed(42)
random.seed(42)
torch.manual_seed(42)

STOCKS = 3
STATE_SIZE = 6
ACTION_SIZE = 3

EPISODES = 100
STEPS = 100

GAMMA = 0.99
TAU = 0.005

ACTOR_LR = 0.001
CRITIC_LR = 0.002

BATCH_SIZE = 64
BUFFER_SIZE = 10000

RISK_PENALTY = 0.1
TRANSACTION_COST = 0.001

class PortfolioEnvironment:

    def __init__(self):

        self.stocks = STOCKS
        self.steps = STEPS

        self.returns = np.random.normal(
            0.001,
            0.02,
            (self.steps, self.stocks)
        )

        self.reset()

    def reset(self):

        self.current_step = 0

        self.portfolio = np.ones(
            self.stocks
```

```
) / self.stocks

return self.get_state()

def get_state(self):

    current_returns = self.returns[
        self.current_step
    ]

    state = np.concatenate(
        [
            current_returns,
            self.portfolio
        ]
    )

    return state.astype(np.float32)

def step(self, action):

    action = np.maximum(action, 0)

    total = np.sum(action)

    if total == 0:
        action = np.ones(
            self.stocks
        ) / self.stocks

    else:
        action = action / total

    market_return = self.returns[
        self.current_step
    ]

    portfolio_return = np.dot(
        action,
        market_return
    )

    risk = np.std(
        action * market_return
    )

    transaction = np.sum(
        np.abs(
            action - self.portfolio
        )
    )

    reward = (
```

```
        portfolio_return
        - RISK_PENALTY * risk
        - TRANSACTION_COST * transaction
    )

    self.portfolio = action

    self.current_step += 1

    done = (
        self.current_step >=
        self.steps - 1
    )

    if not done:

        next_state = self.get_state()

    else:

        next_state = np.zeros(
            STATE_SIZE,
            dtype=np.float32
        )

    return (
        next_state,
        reward,
        done,
        portfolio_return,
        risk
    )

class Actor(nn.Module):

    def __init__(self):

        super().__init__()

        self.model = nn.Sequential(
            nn.Linear(STATE_SIZE, 128),
            nn.ReLU(),

            nn.Linear(128, 128),
            nn.ReLU(),

            nn.Linear(128, ACTION_SIZE),
            nn.Softmax(dim=-1)
        )

    def forward(self, state):
```

```
        return self.model(state)

class Critic(nn.Module):

    def __init__(self):

        super().__init__()

        self.model = nn.Sequential(
            nn.Linear(
                STATE_SIZE + ACTION_SIZE,
                128
            ),

            nn.ReLU(),

            nn.Linear(128, 128),

            nn.ReLU(),

            nn.Linear(128, 1)
        )

    def forward(
        self,
        state,
        action
    ):

        x = torch.cat(
            [state, action],
            dim=1
        )

        return self.model(x)

class ReplayBuffer:

    def __init__(self, size):

        self.buffer = deque(
            maxlen=size
        )

    def add(
        self,
        state,
        action,
        reward,
        next_state,
        done
```

```

):

    self.buffer.append(
        (
            state,
            action,
            reward,
            next_state,
            done
        )
    )

def sample(self, batch_size):

    batch = random.sample(
        self.buffer,
        batch_size
    )

    states, actions, rewards, next_states, dones = zip(
        *batch
    )

    return (
        np.array(states),
        np.array(actions),
        np.array(rewards),
        np.array(next_states),
        np.array(dones)
    )

def __len__(self):

    return len(self.buffer)

def soft_update(
    target,
    source
):

    for target_parameter, source_parameter in zip(
        target.parameters(),
        source.parameters()
    ):

        target_parameter.data.copy_(
            TAU * source_parameter.data
            +
            (1 - TAU)
            * target_parameter.data
        )

```

```
def train_ddpg():

    actor = Actor()

    target_actor = Actor()

    critic = Critic()

    target_critic = Critic()

    target_actor.load_state_dict(
        actor.state_dict()
    )

    target_critic.load_state_dict(
        critic.state_dict()
    )

    actor_optimizer = optim.Adam(
        actor.parameters(),
        lr=ACTOR_LR
    )

    critic_optimizer = optim.Adam(
        critic.parameters(),
        lr=CRITIC_LR
    )

    buffer = ReplayBuffer(
        BUFFER_SIZE
    )

    episode_rewards = []

    episode_returns = []

    episode_risks = []

    for episode in range(EPISODES):

        env = PortfolioEnvironment()

        state = env.reset()

        total_reward = 0

        total_return = 0

        risks = []

        for step in range(STEPS):
```

```
state_tensor = torch.FloatTensor(
    state
).unsqueeze(0)

with torch.no_grad():

    action = actor(
        state_tensor
    ).numpy()[0]

noise = np.random.normal(
    0,
    0.05,
    ACTION_SIZE
)

action = action + noise

action = np.maximum(
    action,
    0
)

if np.sum(action) == 0:

    action = np.ones(
        ACTION_SIZE
    ) / ACTION_SIZE

else:

    action = action / np.sum(
        action
    )

(
    next_state,
    reward,
    done,
    portfolio_return,
    risk
) = env.step(action)

buffer.add(
    state,
    action,
    reward,
    next_state,
    done
)

state = next_state
```

```
total_reward += reward

total_return += portfolio_return

risks.append(risk)

if len(buffer) >= BATCH_SIZE:

    (
        states,
        actions,
        rewards,
        next_states,
        dones
    ) = buffer.sample(
        BATCH_SIZE
    )

    states = torch.FloatTensor(
        states
    )

    actions = torch.FloatTensor(
        actions
    )

    rewards = torch.FloatTensor(
        rewards
    ).unsqueeze(1)

    next_states = torch.FloatTensor(
        next_states
    )

    dones = torch.FloatTensor(
        dones
    ).unsqueeze(1)

    with torch.no_grad():

        next_actions = target_actor(
            next_states
        )

        next_q = target_critic(
            next_states,
            next_actions
        )

        target_q = rewards + GAMMA * (
            1 - dones
        ) * next_q
```

```
current_q = critic(
    states,
    actions
)

critic_loss = nn.functional.mse_loss(
    current_q,
    target_q
)

critic_optimizer.zero_grad()

critic_loss.backward()

critic_optimizer.step()

predicted_actions = actor(
    states
)

actor_loss = -critic(
    states,
    predicted_actions
).mean()

actor_optimizer.zero_grad()

actor_loss.backward()

actor_optimizer.step()

soft_update(
    target_actor,
    actor
)

soft_update(
    target_critic,
    critic
)

if done:

    break

episode_rewards.append(
    total_reward
)

episode_returns.append(
    total_return
)
```

```

        episode_risks.append(
            np.mean(risks)
        )

    if (episode + 1) % 10 == 0:

        print(
            "Episode:",
            episode + 1,
            "| Reward:",
            round(total_reward, 4),
            "| Return:",
            round(total_return, 4),
            "| Risk:",
            round(
                np.mean(risks),
                4
            )
        )

    return (
        actor,
        episode_rewards,
        episode_returns,
        episode_risks
    )

print("=====")
print("DDPG PORTFOLIO OPTIMIZATION")
print("=====")

(
    trained_actor,
    rewards,
    returns,
    risks
) = train_ddpg()

print("\n=====")
print("EVALUATION")
print("=====")

environment = PortfolioEnvironment()

state = environment.reset()

portfolio_values = [1.0]

evaluation_returns = []

for step in range(STEPS - 1):

```

```
state_tensor = torch.FloatTensor(
    state
).unsqueeze(0)

with torch.no_grad():

    action = trained_actor(
        state_tensor
    ).numpy()[0]

action = np.maximum(
    action,
    0
)

action = action / np.sum(
    action
)

(
    next_state,
    reward,
    done,
    portfolio_return,
    risk
) = environment.step(
    action
)

new_value = (
    portfolio_values[-1]
    * (1 + portfolio_return)
)

portfolio_values.append(
    new_value
)

evaluation_returns.append(
    portfolio_return
)

state = next_state

if done:

    break

cumulative_return = (
    portfolio_values[-1] - 1
) * 100
```

```
risk_value = np.std(
    evaluation_returns
)

print(
    "Cumulative Return:",
    round(
        cumulative_return,
        2
    ),
    "%"
)

print(
    "Portfolio Risk:",
    round(
        risk_value,
        4
    )
)

print(
    "Final Portfolio Value:",
    round(
        portfolio_values[-1],
        4
    )
)

print("\nFinal Portfolio Allocation:")

final_state = torch.FloatTensor(
    state
).unsqueeze(0)

with torch.no_grad():

    final_action = trained_actor(
        final_state
    ).numpy()[0]

print(
    "Stock 1:",
    round(final_action[0] * 100, 2),
    "%"
)

print(
    "Stock 2:",
    round(final_action[1] * 100, 2),
    "%"
)
```

```
print(
    "Stock 3:",
    round(final_action[2] * 100, 2),
    "%"
)
```

```
plt.figure(
    figsize=(10, 5)
)
```

```
plt.plot(
    rewards
)
```

```
plt.xlabel(
    "Episode"
)
```

```
plt.ylabel(
    "Reward"
)
```

```
plt.title(
    "DDPG Training Convergence"
)
```

```
plt.grid()
```

```
plt.show()
```

```
plt.figure(
    figsize=(10, 5)
)
```

```
plt.plot(
    np.cumsum(returns)
)
```

```
plt.xlabel(
    "Episode"
)
```

```
plt.ylabel(
    "Cumulative Return"
)
```

```
plt.title(
    "Cumulative Portfolio Return"
)
```

```
plt.grid()

plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    risks
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Risk"
)

plt.title(
    "Portfolio Risk During Training"
)

plt.grid()

plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    portfolio_values
)

plt.xlabel(
    "Trading Step"
)

plt.ylabel(
    "Portfolio Value"
)

plt.title(
    "Final Portfolio Performance"
)

plt.grid()

plt.show()
```

```
=====
DDPG PORTFOLIO OPTIMIZATION
=====
```

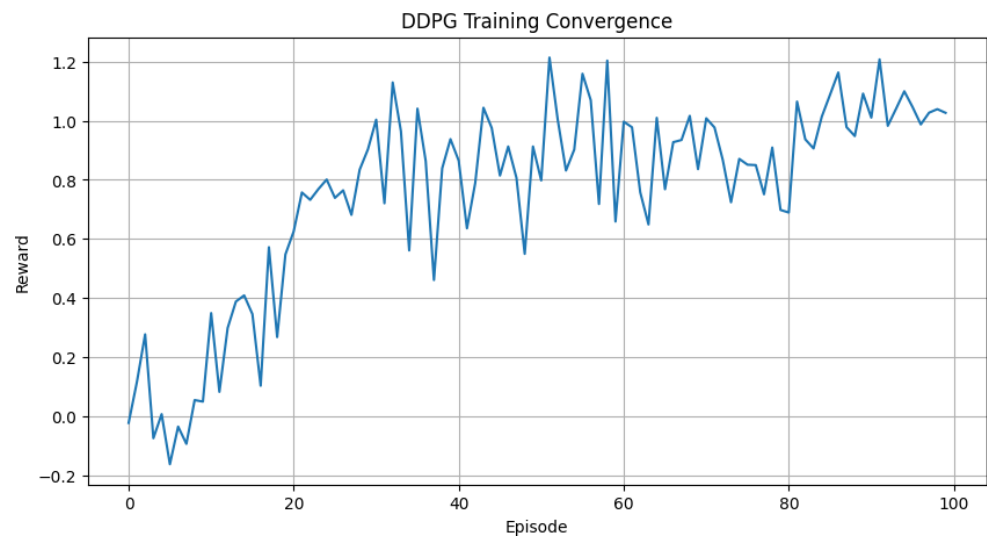
```
Episode: 10 | Reward: 0.0492 | Return: 0.1312 | Risk: 0.0067
Episode: 20 | Reward: 0.5474 | Return: 0.653 | Risk: 0.0056
Episode: 30 | Reward: 0.9033 | Return: 1.0558 | Risk: 0.0071
Episode: 40 | Reward: 0.9377 | Return: 1.0972 | Risk: 0.0069
Episode: 50 | Reward: 0.9128 | Return: 1.075 | Risk: 0.0076
Episode: 60 | Reward: 0.6588 | Return: 0.8217 | Risk: 0.006
Episode: 70 | Reward: 0.8362 | Return: 0.9978 | Risk: 0.0067
Episode: 80 | Reward: 0.698 | Return: 0.867 | Risk: 0.0067
Episode: 90 | Reward: 1.0918 | Return: 1.2736 | Risk: 0.0076
Episode: 100 | Reward: 1.0268 | Return: 1.2088 | Risk: 0.008
```

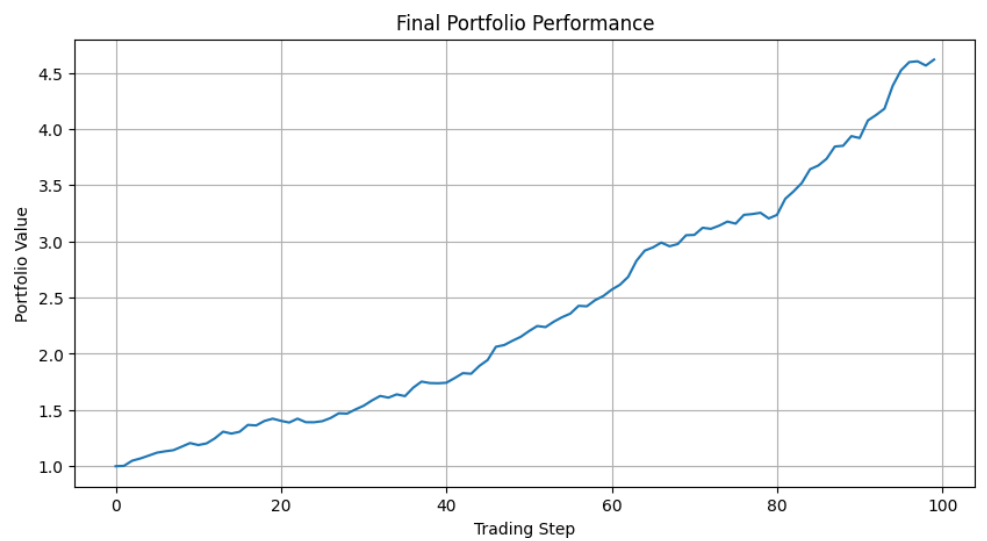
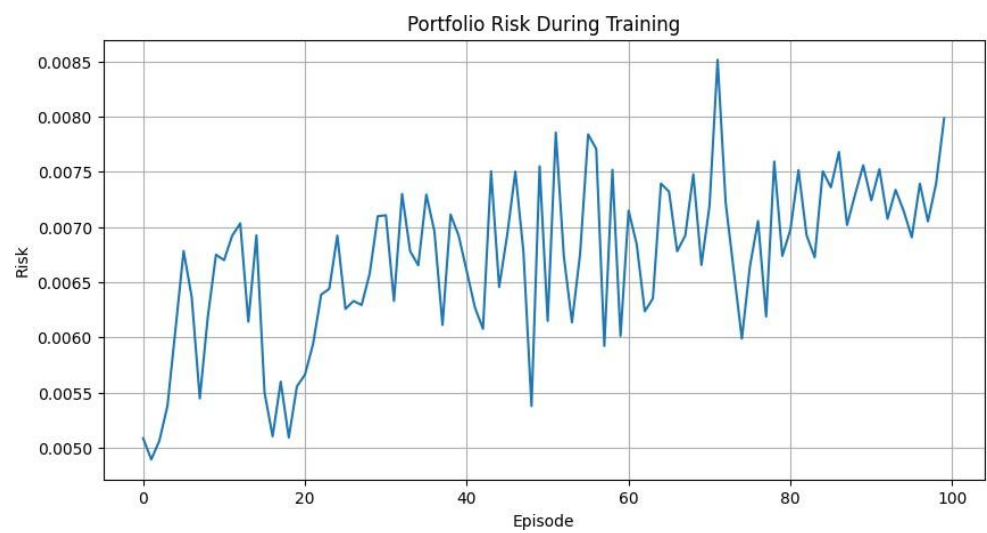
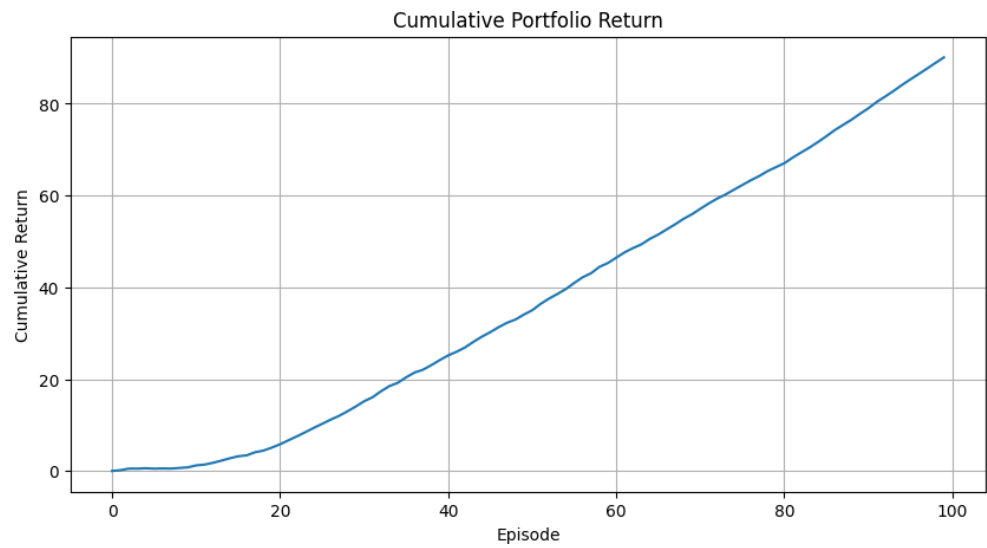
```
=====
EVALUATION
=====
```

```
Cumulative Return: 361.81 %
Portfolio Risk: 0.017
Final Portfolio Value: 4.6181
```

Final Portfolio Allocation:

```
Stock 1: 3.49 %
Stock 2: 96.28 %
Stock 3: 0.24 %
```





4. Design and develop a Proximal Policy Optimization (PPO) model for controlling Smart HVAC Energy Management systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

```
In [10]: import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

EPISODES = 200
STEPS = 50

GAMMA = 0.99
LEARNING_RATE = 0.001

CLIP_EPSILON = 0.2

class HVACEnvironment:

    def __init__(self):

        self.comfort_min = 21
        self.comfort_max = 24

        self.reset()

    def reset(self):

        self.indoor_temp = random.uniform(
            21,
            27
        )

        self.outdoor_temp = random.uniform(
            28,
            35
        )

        self.occupancy = random.randint(
            5,
            30
        )

        self.hvac_level = 1

        return self.get_state()

    def get_state(self):

        return np.array([
```

```
        self.indoor_temp / 40,  
        self.outdoor_temp / 40,  
        self.occupancy / 30,  
        self.hvac_level / 3  
    ], dtype=np.float32)  
  
def step(self, action):  
  
    if action == 0:  
  
        self.hvac_level -= 1  
  
    elif action == 1:  
  
        self.hvac_level = self.hvac_level  
  
    elif action == 2:  
  
        self.hvac_level += 1  
  
    self.hvac_level = max(  
        0,  
        min(3, self.hvac_level)  
    )  
  
    outside_effect = (  
        self.outdoor_temp -  
        self.indoor_temp  
    ) * 0.05  
  
    cooling_effect = (  
        self.hvac_level * 0.5  
    )  
  
    self.indoor_temp += (  
        outside_effect -  
        cooling_effect  
    )  
  
    self.indoor_temp += random.uniform(  
        -0.1,  
        0.1  
    )  
  
    energy = (  
        1 +  
        self.hvac_level * 2 +  
        self.occupancy * 0.03  
    )  
  
    if (  
        self.indoor_temp >=  
        self.comfort_min
```

```
        and
        self.indoor_temp <=
        self.comfort_max
    ):

        comfort_reward = 5

    else:

        distance = min(
            abs(
                self.indoor_temp -
                self.comfort_min
            ),
            abs(
                self.indoor_temp -
                self.comfort_max
            )
        )

        comfort_reward = -5 * distance

    reward = (
        comfort_reward -
        0.5 * energy
    )

    self.outdoor_temp += random.uniform(
        -0.3,
        0.3
    )

    self.outdoor_temp = max(
        25,
        min(
            40,
            self.outdoor_temp
        )
    )

    self.occupancy += random.randint(
        -2,
        2
    )

    self.occupancy = max(
        0,
        min(
            30,
            self.occupancy
        )
    )
)
```

```
        next_state = self.get_state()

        return (
            next_state,
            reward,
            energy
        )

class PolicyNetwork(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(4, 64),

            nn.ReLU(),

            nn.Linear(64, 64),

            nn.ReLU(),

            nn.Linear(64, 3),

            nn.Softmax(dim=-1)
        )

    def forward(self, state):

        return self.network(state)

class ValueNetwork(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(4, 64),

            nn.ReLU(),

            nn.Linear(64, 64),

            nn.ReLU(),

            nn.Linear(64, 1)
        )
```

```
def forward(self, state):

    return self.network(state)

def train_reinforce():

    env = HVACEnvironment()

    policy = PolicyNetwork()

    optimizer = optim.Adam(
        policy.parameters(),
        lr=LEARNING_RATE
    )

    rewards_history = []

    energy_history = []

    comfort_history = []

    for episode in range(EPIISODES):

        state = env.reset()

        log_probs = []

        rewards = []

        total_energy = 0

        comfort_count = 0

        for step in range(STEPS):

            state_tensor = torch.FloatTensor(
                state
            )

            probabilities = policy(
                state_tensor
            )

            distribution = torch.distributions.Categorical(
                probabilities
            )

            action = distribution.sample()

            log_prob = distribution.log_prob(
                action
```

```
)

next_state, reward, energy = env.step(
    action.item()
)

log_probs.append(
    log_prob
)

rewards.append(
    reward
)

total_energy += energy

if (
    env.comfort_min <=
    env.indoor_temp <=
    env.comfort_max
):

    comfort_count += 1

state = next_state

returns = []

G = 0

for reward in reversed(
    rewards
):

    G = reward + GAMMA * G

    returns.insert(
        0,
        G
    )

returns = torch.FloatTensor(
    returns
)

if len(returns) > 1:

    returns = (
        returns -
        returns.mean()
    ) / (
        returns.std() +
        1e-8
```

```
    )

    loss = 0

    for log_prob, G in zip(
        log_probs,
        returns
    ):

        loss += -log_prob * G

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    rewards_history.append(
        sum(rewards)
    )

    energy_history.append(
        total_energy
    )

    comfort_history.append(
        comfort_count / STEPS * 100
    )

    return (
        policy,
        rewards_history,
        energy_history,
        comfort_history
    )

def train_ppo():

    env = HVACEnvironment()

    policy = PolicyNetwork()

    value = ValueNetwork()

    policy_optimizer = optim.Adam(
        policy.parameters(),
        lr=LEARNING_RATE
    )

    value_optimizer = optim.Adam(
        value.parameters(),
        lr=LEARNING_RATE
```

```
)

rewards_history = []

energy_history = []

comfort_history = []

for episode in range(EPISODES):

    state = env.reset()

    states = []

    actions = []

    old_log_probs = []

    rewards = []

    total_energy = 0

    comfort_count = 0

    for step in range(STEPS):

        state_tensor = torch.FloatTensor(
            state
        )

        probabilities = policy(
            state_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        action = distribution.sample()

        log_prob = distribution.log_prob(
            action
        )

        next_state, reward, energy = env.step(
            action.item()
        )

        states.append(state)

        actions.append(
            action.item()
        )
```

```
        old_log_probs.append(
            log_prob.item()
        )

        rewards.append(
            reward
        )

        total_energy += energy

        if (
            env.comfort_min <=
            env.indoor_temp <=
            env.comfort_max
        ):

            comfort_count += 1

        state = next_state

    returns = []

    G = 0

    for reward in reversed(
        rewards
    ):

        G = reward + GAMMA * G

        returns.insert(
            0,
            G
        )

    states_tensor = torch.FloatTensor(
        np.array(states)
    )

    actions_tensor = torch.LongTensor(
        actions
    )

    old_log_probs_tensor = torch.FloatTensor(
        old_log_probs
    )

    returns_tensor = torch.FloatTensor(
        returns
    )

    values = value(
```

```
        states_tensor
    ).squeeze()

    advantages = (
        returns_tensor -
        values.detach()
    )

    if len(advantages) > 1:

        advantages = (
            advantages -
            advantages.mean()
        ) / (
            advantages.std() +
            1e-8
        )

    for _ in range(4):

        probabilities = policy(
            states_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        new_log_probs = distribution.log_prob(
            actions_tensor
        )

        ratio = torch.exp(
            new_log_probs -
            old_log_probs_tensor
        )

        clipped_ratio = torch.clamp(
            ratio,
            1 - CLIP_EPSILON,
            1 + CLIP_EPSILON
        )

        policy_loss = -torch.min(
            ratio * advantages,
            clipped_ratio * advantages
        ).mean()

        policy_optimizer.zero_grad()

        policy_loss.backward()

        policy_optimizer.step()
```

```

        values = value(
            states_tensor
        ).squeeze()

        value_loss = nn.functional.mse_loss(
            values,
            returns_tensor
        )

        value_optimizer.zero_grad()

        value_loss.backward()

        value_optimizer.step()

    rewards_history.append(
        sum(rewards)
    )

    energy_history.append(
        total_energy
    )

    comfort_history.append(
        comfort_count / STEPS * 100
    )

    return (
        policy,
        rewards_history,
        energy_history,
        comfort_history
    )

print("=====")
print("SMART HVAC ENERGY MANAGEMENT")
print("PPO vs REINFORCE")
print("=====")

print("\nTraining REINFORCE...")

(
    reinforce_policy,
    reinforce_rewards,
    reinforce_energy,
    reinforce_comfort
) = train_reinforce()

print("REINFORCE Training Completed")

print("\nTraining PPO...")

```

```

(
    ppo_policy,
    ppo_rewards,
    ppo_energy,
    ppo_comfort
) = train_ppo()

print("PPO Training Completed")

window = 30

reinforce_avg_reward = np.mean(
    reinforce_rewards[-window:]
)

ppo_avg_reward = np.mean(
    ppo_rewards[-window:]
)

reinforce_avg_energy = np.mean(
    reinforce_energy[-window:]
)

ppo_avg_energy = np.mean(
    ppo_energy[-window:]
)

reinforce_avg_comfort = np.mean(
    reinforce_comfort[-window:]
)

ppo_avg_comfort = np.mean(
    ppo_comfort[-window:]
)

print("\n=====")
print("FINAL RESULTS")
print("=====")

print("\nREINFORCE")

print(
    "Average Reward:",
    round(
        reinforce_avg_reward,
        2
    )
)

print(

```

```

    "Average Energy:",
    round(
        reinforce_avg_energy,
        2
    )
)

print(
    "Comfort Percentage:",
    round(
        reinforce_avg_comfort,
        2
    ),
    "%"
)

print("\nPPO")

print(
    "Average Reward:",
    round(
        ppo_avg_reward,
        2
    )
)

print(
    "Average Energy:",
    round(
        ppo_avg_energy,
        2
    )
)

print(
    "Comfort Percentage:",
    round(
        ppo_avg_comfort,
        2
    ),
    "%"
)

print("\n=====")
print("COMPARISON")
print("=====")

print(
    "\nMetric".ljust(25),
    "REINFORCE".ljust(15),
    "PPO"

```

```
)

print(
    "Average Reward".ljust(25),
    round(
        reinforce_avg_reward,
        2
    ),
    " " * 8,
    round(
        ppo_avg_reward,
        2
    )
)

print(
    "Average Energy".ljust(25),
    round(
        reinforce_avg_energy,
        2
    ),
    " " * 8,
    round(
        ppo_avg_energy,
        2
    )
)

print(
    "Comfort (%)".ljust(25),
    round(
        reinforce_avg_comfort,
        2
    ),
    " " * 8,
    round(
        ppo_avg_comfort,
        2
    )
)

if ppo_avg_energy < reinforce_avg_energy:
    print(
        "\nPPO uses less energy."
    )

else:
    print(
        "\nREINFORCE uses less energy."
    )
```

```
if ppo_avg_comfort > reinforce_avg_comfort:
    print(
        "PPO provides better occupant comfort."
    )

else:
    print(
        "REINFORCE provides better occupant comfort."
    )

if ppo_avg_reward > reinforce_avg_reward:
    print(
        "PPO achieves higher reward."
    )

else:
    print(
        "REINFORCE achieves higher reward."
    )

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    reinforce_rewards,
    label="REINFORCE"
)

plt.plot(
    ppo_rewards,
    label="PPO"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Reward"
)

plt.title(
    "Training Reward Comparison"
)

plt.legend()

plt.grid()

plt.show()
```

```
plt.figure(
    figsize=(10, 5)
)

plt.plot(
    reinforce_energy,
    label="REINFORCE"
)

plt.plot(
    ppo_energy,
    label="PPO"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Energy Consumption"
)

plt.title(
    "HVAC Energy Consumption"
)

plt.legend()

plt.grid()

plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    reinforce_comfort,
    label="REINFORCE"
)

plt.plot(
    ppo_comfort,
    label="PPO"
)

plt.xlabel(
    "Episode"
)
```

```

plt.ylabel(
    "Comfort Percentage"
)

plt.title(
    "Occupant Comfort Comparison"
)

plt.legend()

plt.grid()

plt.show()

print("\n=====")
print("PROJECT COMPLETED")
print("=====")

```

```

=====
SMART HVAC ENERGY MANAGEMENT
PPO vs REINFORCE
=====

```

```

Training REINFORCE...
REINFORCE Training Completed

```

```

Training PPO...
PPO Training Completed

```

```

=====
FINAL RESULTS
=====

```

```

REINFORCE
Average Reward: -463.67
Average Energy: 162.14
Comfort Percentage: 29.6 %

```

```

PPO
Average Reward: -245.53
Average Energy: 166.69
Comfort Percentage: 38.4 %

```

```

=====
COMPARISON
=====

```

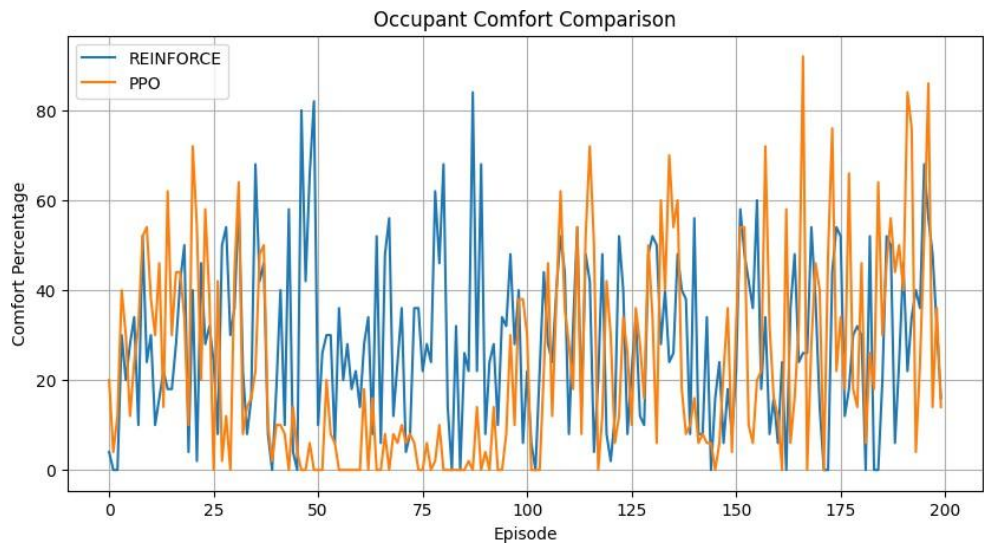
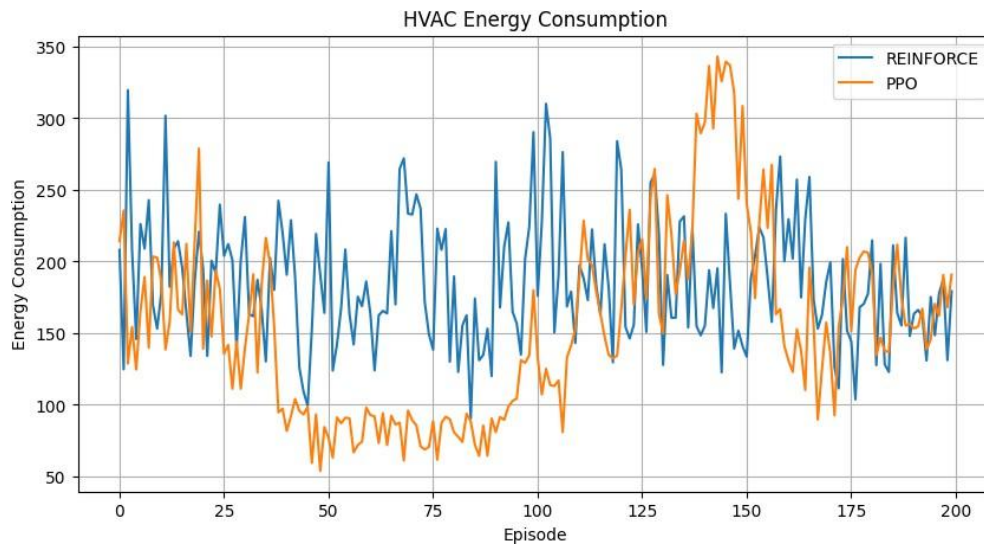
Metric	REINFORCE	PPO
Average Reward	-463.67	-245.53
Average Energy	162.14	166.69
Comfort (%)	29.6	38.4

```

REINFORCE uses less energy.

```

PPO provides better occupant comfort.
PPO achieves higher reward.



=====

PROJECT COMPLETED

=====

5. Implement a Trust Region Policy Optimization (TRPO) algorithm for controlling a Industrial robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.

```
In [11]: import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

EPISODES = 250
MAX_STEPS = 40

GAMMA = 0.99
LEARNING_RATE = 0.001

KL_LIMIT = 0.02

class RoboticArmEnvironment:

    def __init__(self):

        self.reset()

    def reset(self):

        self.arm_x = random.uniform(
            0,
            1
        )

        self.arm_y = random.uniform(
            0,
            1
        )

        self.target_x = random.uniform(
            0.2,
            0.8
        )

        self.target_y = random.uniform(
            0.2,
            0.8
        )

        return self.get_state()

    def get_state(self):

        return np.array(
```

```
[
    self.arm_x,
    self.arm_y,
    self.target_x,
    self.target_y
],
dtype=np.float32
)

def distance(self):

    return np.sqrt(
        (self.arm_x - self.target_x) ** 2
        +
        (self.arm_y - self.target_y) ** 2
    )

def step(self, action):

    old_distance = self.distance()

    movement = 0.05

    if action == 0:

        self.arm_y += movement

    elif action == 1:

        self.arm_y -= movement

    elif action == 2:

        self.arm_x -= movement

    elif action == 3:

        self.arm_x += movement

    self.arm_x = np.clip(
        self.arm_x,
        0,
        1
    )

    self.arm_y = np.clip(
        self.arm_y,
        0,
        1
    )

    new_distance = self.distance()
```

```
improvement = (
    old_distance -
    new_distance
)

reward = improvement * 20

reward -= 0.05

done = False

if new_distance < 0.05:

    reward += 20

    done = True

return (
    self.get_state(),
    reward,
    done
)

class PolicyNetwork(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(
                4,
                64
            ),

            nn.Tanh(),

            nn.Linear(
                64,
                64
            ),

            nn.Tanh(),

            nn.Linear(
                64,
                4
            )
        )

    def forward(self, state):
```

```
        return torch.softmax(
            self.network(state),
            dim=-1
        )

class ValueNetwork(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(
                4,
                64
            ),

            nn.Tanh(),

            nn.Linear(
                64,
                64
            ),

            nn.Tanh(),

            nn.Linear(
                64,
                1
            )
        )

    def forward(self, state):

        return self.network(state)

def calculate_returns(rewards):

    returns = []

    G = 0

    for reward in reversed(
        rewards
    ):

        G = reward + GAMMA * G

        returns.insert(
```

```
        0,
        G
    )

    return torch.FloatTensor(
        returns
    )

def calculate_kl(
    old_prob,
    new_prob
):

    old_log = torch.log(
        old_prob + 1e-8
    )

    new_log = torch.log(
        new_prob + 1e-8
    )

    kl = torch.sum(
        old_prob *
        (old_log - new_log),
        dim=1
    )

    return kl.mean()

def train_trpo():

    env = RoboticArmEnvironment()

    policy = PolicyNetwork()

    value = ValueNetwork()

    policy_optimizer = optim.Adam(
        policy.parameters(),
        lr=LEARNING_RATE
    )

    value_optimizer = optim.Adam(
        value.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []

    precision_history = []
```

```
steps_history = []

success_history = []

stability_history = []

for episode in range(
    EPISODES
):

    state = env.reset()

    states = []

    actions = []

    old_probabilities = []

    rewards = []

    success = 0

    for step in range(
        MAX_STEPS
    ):

        state_tensor = torch.FloatTensor(
            state
        ).unsqueeze(0)

        probabilities = policy(
            state_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        action = distribution.sample()

        action_probability = probabilities[
            0,
            action.item()
        ]

        next_state, reward, done = env.step(
            action.item()
        )

        states.append(
            state
        )
```

```
actions.append(
    action.item()
)

old_probabilities.append(
    probabilities.detach()
)

rewards.append(
    reward
)

state = next_state

if done:

    success = 1

    break

states_tensor = torch.FloatTensor(
    np.array(states)
)

actions_tensor = torch.LongTensor(
    actions
)

old_probabilities_tensor = torch.cat(
    old_probabilities
)

returns = calculate_returns(
    rewards
)

values = value(
    states_tensor
).squeeze()

advantages = (
    returns -
    values.detach()
)

if len(advantages) > 1:

    advantages = (
        advantages -
        advantages.mean()
    ) / (
        advantages.std() +
        1e-8
```

```
)

old_action_prob = old_probabilities_tensor[
    range(len(actions)),
    actions_tensor
]

for _ in range(5):

    new_probabilities = policy(
        states_tensor
    )

    new_action_prob = new_probabilities[
        range(len(actions)),
        actions_tensor
    ]

    ratio = (
        new_action_prob /
        (old_action_prob + 1e-8)
    )

    surrogate = (
        ratio *
        advantages
    ).mean()

    kl = calculate_kl(
        old_probabilities_tensor,
        new_probabilities
    )

    if kl.item() > KL_LIMIT:

        break

    loss = -surrogate

    policy_optimizer.zero_grad()

    loss.backward()

    policy_optimizer.step()

new_values = value(
    states_tensor
).squeeze()

value_loss = nn.functional.mse_loss(
    new_values,
    returns
)
```

```
value_optimizer.zero_grad()

value_loss.backward()

value_optimizer.step()

final_distance = env.distance()

precision = (
    1 /
    (1 + final_distance)
)

reward_history.append(
    sum(rewards)
)

precision_history.append(
    precision
)

steps_history.append(
    step + 1
)

success_history.append(
    success
)

stability_history.append(
    kl.item()
)

return (
    policy,
    reward_history,
    precision_history,
    steps_history,
    success_history,
    stability_history
)

print("=====")
print("TRPO INDUSTRIAL ROBOTIC ARM")
print("=====")

print("\nTraining TRPO...")

(
    policy,
    rewards,
```

```
        precision,
        steps,
        success,
        stability
    ) = train_trpo()

print("\nTraining Completed")

window = 30

average_reward = np.mean(
    rewards[-window:]
)

average_precision = np.mean(
    precision[-window:]
)

average_steps = np.mean(
    steps[-window:]
)

success_rate = (
    np.mean(
        success[-window:]
    ) * 100
)

average_kl = np.mean(
    stability[-window:]
)

print("\n=====")
print("FINAL PERFORMANCE")
print("=====")

print(
    "Average Reward:",
    round(
        average_reward,
        3
    )
)

print(
    "Average Precision:",
    round(
        average_precision,
        3
    )
)
```

```

print(
    "Average Task Steps:",
    round(
        average_steps,
        2
    )
)

print(
    "Task Success Rate:",
    round(
        success_rate,
        2
    ),
    "%"
)

print(
    "Average KL Divergence:",
    round(
        average_kl,
        5
    )
)

print("\n=====")
print("PRODUCTION EFFICIENCY")
print("=====")

efficiency = (
    success_rate /
    average_steps
)

print(
    "Production Efficiency:",
    round(
        efficiency,
        3
    )
)

print("\n=====")
print("POLICY STABILITY")
print("=====")

if average_kl <= KL_LIMIT:

    print(
        "Policy is stable."
    )

```

```
)

else:

    print(
        "Policy requires further training."
    )

print("\n=====")
print("PRECISION")
print("=====")

if average_precision > 0.95:

    print(
        "Excellent robotic precision."
    )

elif average_precision > 0.85:

    print(
        "Good robotic precision."
    )

else:

    print(
        "Precision needs improvement."
    )

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    rewards
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Reward"
)

plt.title(
    "TRPO Training Reward"
)

plt.grid()
```

```
plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    precision
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Precision"
)

plt.title(
    "Robotic Arm Precision"
)

plt.grid()

plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    steps
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Steps"
)

plt.title(
    "Task Completion Speed"
)

plt.grid()

plt.show()
```

```
plt.figure(
    figsize=(10, 5)
)

plt.plot(
    stability
)

plt.axhline(
    KL_LIMIT,
    linestyle="--",
    label="TRPO KL Limit"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "KL Divergence"
)

plt.title(
    "Policy Stability"
)

plt.legend()

plt.grid()

plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    np.cumsum(success)
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Successful Tasks"
)

plt.title(
    "Production Efficiency"
)
```

```
plt.grid()

plt.show()

print("\n=====")
print("PROJECT COMPLETED")
print("=====")
```

```
=====
TRPO INDUSTRIAL ROBOTIC ARM
=====
```

Training TRPO...

Training Completed

```
=====
FINAL PERFORMANCE
=====
```

Average Reward: 13.335
Average Precision: 0.903
Average Task Steps: 30.03
Task Success Rate: 43.33 %
Average KL Divergence: 0.01076

```
=====
PRODUCTION EFFICIENCY
=====
```

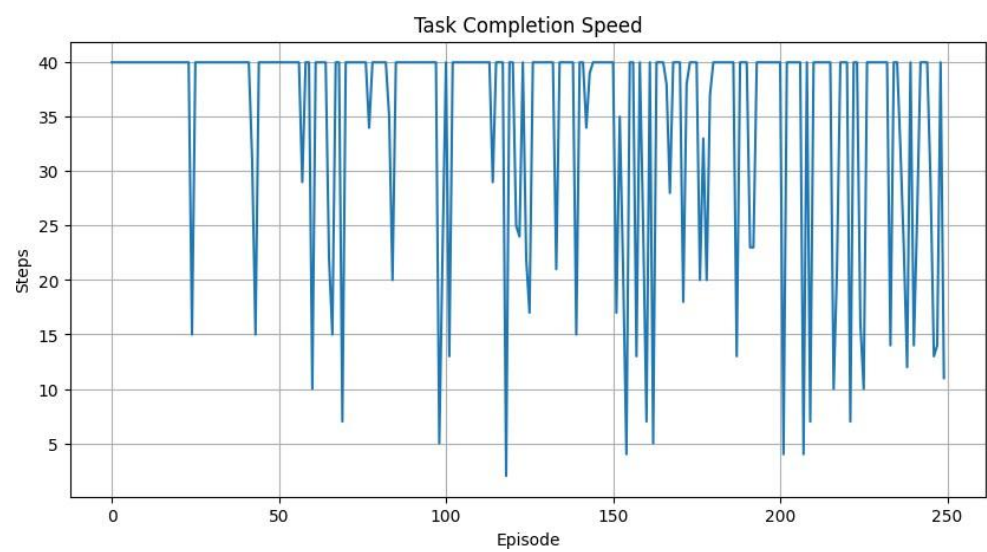
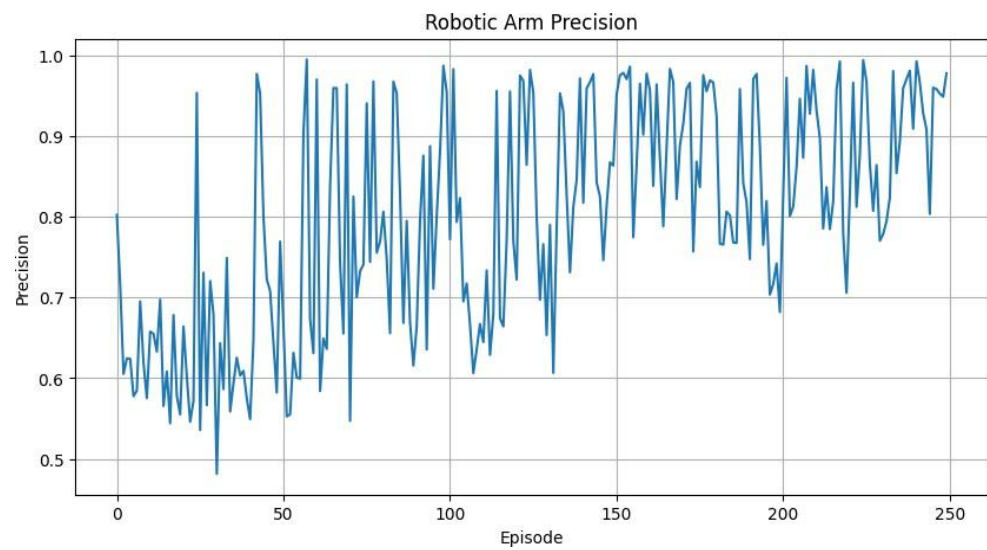
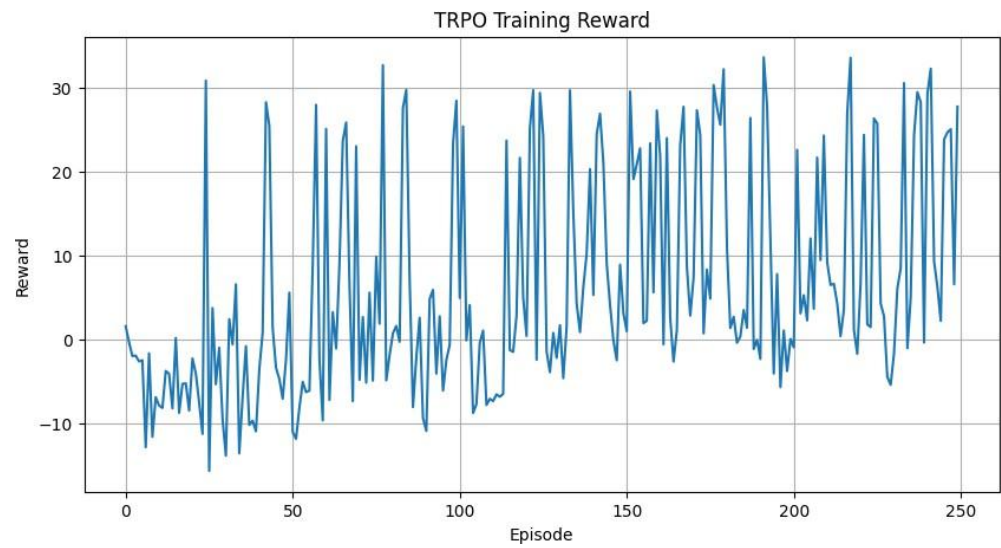
Production Efficiency: 1.443

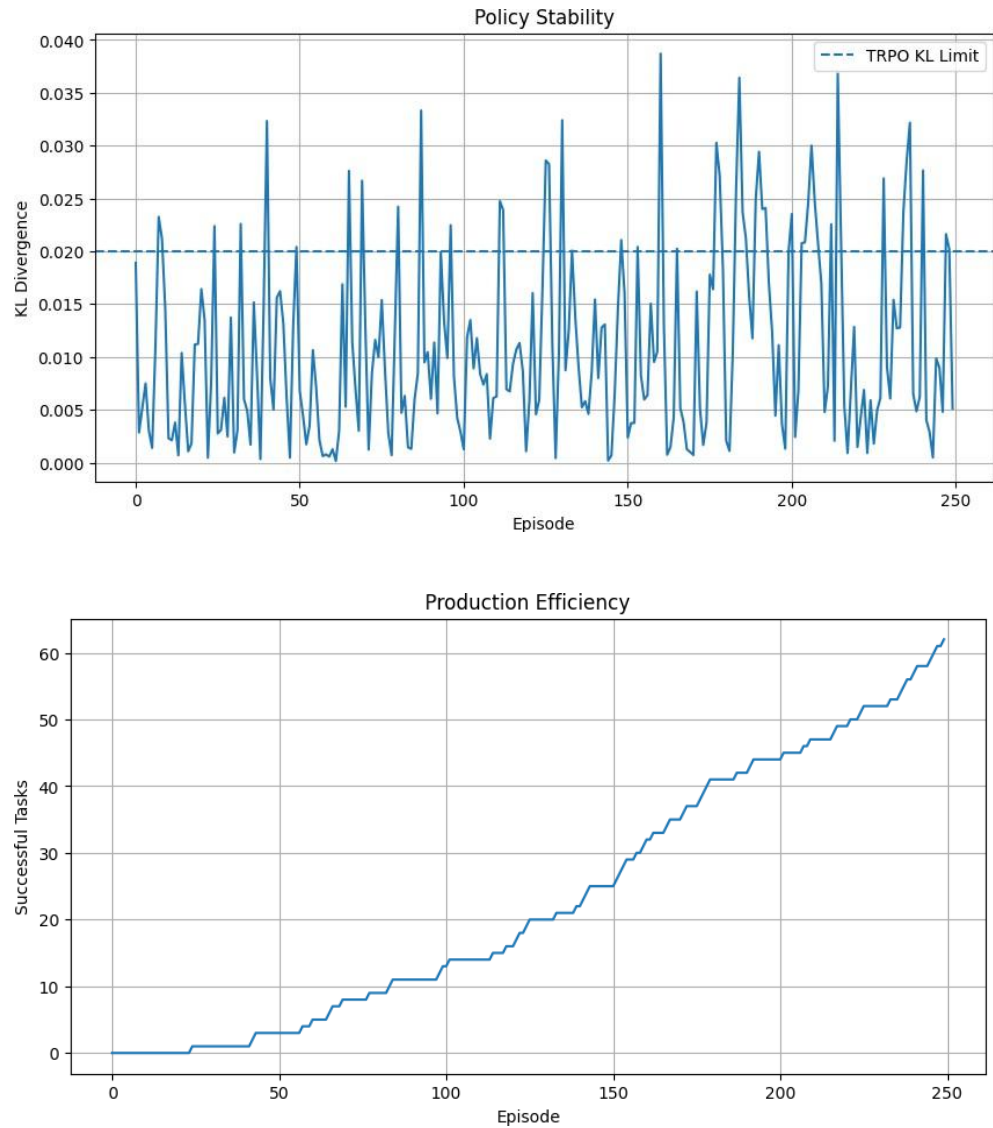
```
=====
POLICY STABILITY
=====
```

Policy is stable.

```
=====
PRECISION
=====
```

Good robotic precision.





=====
PROJECT COMPLETED
=====

6. Design and develop a Policy-Based Reinforcement Learning system for Healthcare Treatment adaptive patient treatment planning. Compare A2C and PPO in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

```
In [12]: import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

EPISODES = 250
STEPS = 30

GAMMA = 0.95
LEARNING_RATE = 0.001
CLIP_EPSILON = 0.2

class PatientEnvironment:

    def __init__(self):
        self.reset()

    def reset(self):

        self.health = random.uniform(0.30, 0.70)
        self.symptoms = random.uniform(0.30, 0.70)
        self.risk = random.uniform(0.10, 0.50)

        return self.get_state()

    def get_state(self):

        return np.array([
            self.health,
            self.symptoms,
            self.risk
        ], dtype=np.float32)

    def step(self, action):

        old_health = self.health
        old_symptoms = self.symptoms

        if action == 0:
            health_change = 0.01
            symptom_change = -0.01
            treatment_cost = 0.02

        elif action == 1:
            health_change = 0.04
            symptom_change = -0.04
```

```
treatment_cost = 0.06

else:
    health_change = 0.06
    symptom_change = -0.06
    treatment_cost = 0.12

noise = random.uniform(
    -0.02,
    0.02
)

self.health += (
    health_change +
    noise
)

self.symptoms += (
    symptom_change +
    noise
)

self.health = np.clip(
    self.health,
    0,
    1
)

self.symptoms = np.clip(
    self.symptoms,
    0,
    1
)

improvement = (
    self.health -
    old_health
)

symptom_improvement = (
    old_symptoms -
    self.symptoms
)

reward = (
    5 * improvement
    +
    3 * symptom_improvement
    -
    treatment_cost
)

if self.health > 0.85:
```

```
        reward += 2

    if self.symptoms < 0.15:

        reward += 2

    next_state = self.get_state()

    return (
        next_state,
        reward
    )

class Actor(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(3, 64),
            nn.ReLU(),

            nn.Linear(64, 64),
            nn.ReLU(),

            nn.Linear(64, 3),
            nn.Softmax(dim=-1)
        )

    def forward(self, state):

        return self.network(state)

class Critic(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(3, 64),
            nn.ReLU(),

            nn.Linear(64, 64),
            nn.ReLU(),

            nn.Linear(64, 1)
        )
```

```
def forward(self, state):

    return self.network(state)

def calculate_returns(rewards):

    returns = []

    G = 0

    for reward in reversed(rewards):

        G = reward + GAMMA * G

        returns.insert(
            0,
            G
        )

    return torch.FloatTensor(
        returns
    )

def train_a2c():

    env = PatientEnvironment()

    actor = Actor()
    critic = Critic()

    actor_optimizer = optim.Adam(
        actor.parameters(),
        lr=LEARNING_RATE
    )

    critic_optimizer = optim.Adam(
        critic.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []
    effectiveness_history = []
    stability_history = []

    for episode in range(EPISODES):

        state = env.reset()

        log_probs = []
        values = []
        rewards = []
```

```
for step in range(STEPS):

    state_tensor = torch.FloatTensor(
        state
    ).unsqueeze(0)

    probabilities = actor(
        state_tensor
    )

    distribution = torch.distributions.Categorical(
        probabilities
    )

    action = distribution.sample()

    log_prob = distribution.log_prob(
        action
    )

    value = critic(
        state_tensor
    ).squeeze()

    next_state, reward = env.step(
        action.item()
    )

    log_probs.append(
        log_prob
    )

    values.append(
        value
    )

    rewards.append(
        reward
    )

    state = next_state

returns = calculate_returns(
    rewards
)

values = torch.stack(
    values
)

advantages = (
    returns -
```

```
        values.detach()
    )

    if len(advantages) > 1:

        advantages = (
            advantages -
            advantages.mean()
        ) / (
            advantages.std() +
            1e-8
        )

    actor_loss = 0

    for log_prob, advantage in zip(
        log_probs,
        advantages
    ):

        actor_loss += (
            -log_prob * advantage
        )

    critic_loss = nn.functional.mse_loss(
        values,
        returns
    )

    actor_optimizer.zero_grad()

    actor_loss.backward()

    actor_optimizer.step()

    critic_optimizer.zero_grad()

    critic_loss.backward()

    critic_optimizer.step()

    total_reward = sum(
        rewards
    )

    effectiveness = (
        env.health -
        env.symptoms
    )

    reward_history.append(
        total_reward
    )
```

```
        effectiveness_history.append(
            effectiveness
        )

        stability_history.append(
            abs(
                actor_loss.item()
            )
        )

    return (
        actor,
        reward_history,
        effectiveness_history,
        stability_history
    )

def train_ppo():

    env = PatientEnvironment()

    actor = Actor()
    critic = Critic()

    actor_optimizer = optim.Adam(
        actor.parameters(),
        lr=LEARNING_RATE
    )

    critic_optimizer = optim.Adam(
        critic.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []
    effectiveness_history = []
    stability_history = []

    for episode in range(EPISODES):

        state = env.reset()

        states = []
        actions = []
        old_log_probs = []
        rewards = []

        for step in range(STEPS):

            state_tensor = torch.FloatTensor(
                state
```

```
)unsqueeze(0)

probabilities = actor(
    state_tensor
)

distribution = torch.distributions.Categorical(
    probabilities
)

action = distribution.sample()

log_prob = distribution.log_prob(
    action
)

next_state, reward = env.step(
    action.item()
)

states.append(state)

actions.append(
    action.item()
)

old_log_probs.append(
    log_prob.item()
)

rewards.append(
    reward
)

state = next_state

states_tensor = torch.FloatTensor(
    np.array(states)
)

actions_tensor = torch.LongTensor(
    actions
)

old_log_probs_tensor = torch.FloatTensor(
    old_log_probs
)

returns = calculate_returns(
    rewards
)

values = critic(
```

```
        states_tensor
    ).squeeze()

    advantages = (
        returns -
        values.detach()
    )

    if len(advantages) > 1:

        advantages = (
            advantages -
            advantages.mean()
        ) / (
            advantages.std() +
            1e-8
        )

    for _ in range(4):

        probabilities = actor(
            states_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        new_log_probs = distribution.log_prob(
            actions_tensor
        )

        ratio = torch.exp(
            new_log_probs -
            old_log_probs_tensor
        )

        clipped_ratio = torch.clamp(
            ratio,
            1 - CLIP_EPSILON,
            1 + CLIP_EPSILON
        )

        actor_loss = -torch.min(
            ratio * advantages,
            clipped_ratio * advantages
        ).mean()

        actor_optimizer.zero_grad()

        actor_loss.backward()

        actor_optimizer.step()
```

```
        new_values = critic(
            states_tensor
        ).squeeze()

        critic_loss = nn.functional.mse_loss(
            new_values,
            returns
        )

        critic_optimizer.zero_grad()

        critic_loss.backward()

        critic_optimizer.step()

    total_reward = sum(
        rewards
    )

    effectiveness = (
        env.health -
        env.symptoms
    )

    reward_history.append(
        total_reward
    )

    effectiveness_history.append(
        effectiveness
    )

    stability_history.append(
        abs(
            actor_loss.item()
        )
    )

    return (
        actor,
        reward_history,
        effectiveness_history,
        stability_history
    )

print("=====")
print("ADAPTIVE PATIENT TREATMENT SIMULATION")
print("A2C vs PPO")
print("=====")

print("\nTraining A2C...")
```

```
(
    a2c_actor,
    a2c_rewards,
    a2c_effectiveness,
    a2c_stability
) = train_a2c()

print("A2C Training Completed.")

print("\nTraining PPO...")

(
    ppo_actor,
    ppo_rewards,
    ppo_effectiveness,
    ppo_stability
) = train_ppo()

print("PPO Training Completed.")

window = 30

a2c_reward = np.mean(
    a2c_rewards[-window:]
)

ppo_reward = np.mean(
    ppo_rewards[-window:]
)

a2c_effectiveness_value = np.mean(
    a2c_effectiveness[-window:]
)

ppo_effectiveness_value = np.mean(
    ppo_effectiveness[-window:]
)

a2c_stability_value = np.std(
    a2c_rewards[-window:]
)

ppo_stability_value = np.std(
    ppo_rewards[-window:]
)

print("\n=====")
print("FINAL RESULTS")
print("=====")
```

```
print("\nA2C")

print(
    "Average Cumulative Reward:",
    round(
        a2c_reward,
        3
    )
)

print(
    "Treatment Effectiveness:",
    round(
        a2c_effectiveness_value,
        3
    )
)

print(
    "Reward Standard Deviation:",
    round(
        a2c_stability_value,
        3
    )
)

print("\nPPO")

print(
    "Average Cumulative Reward:",
    round(
        ppo_reward,
        3
    )
)

print(
    "Treatment Effectiveness:",
    round(
        ppo_effectiveness_value,
        3
    )
)

print(
    "Reward Standard Deviation:",
    round(
        ppo_stability_value,
        3
    )
)
```

```
print("\n=====")
print("COMPARISON")
print("=====")

print(
    "\nMetric".ljust(30),
    "A2C".ljust(15),
    "PPO"
)

print(
    "Cumulative Reward".ljust(30),
    round(a2c_reward, 3),
    " " * 8,
    round(ppo_reward, 3)
)

print(
    "Treatment Effectiveness".ljust(30),
    round(a2c_effectiveness_value, 3),
    " " * 8,
    round(ppo_effectiveness_value, 3)
)

print(
    "Reward Std. Deviation".ljust(30),
    round(a2c_stability_value, 3),
    " " * 8,
    round(ppo_stability_value, 3)
)

if ppo_reward > a2c_reward:

    print(
        "\nPPO achieved higher cumulative reward."
    )

else:

    print(
        "\nA2C achieved higher cumulative reward."
    )

if ppo_effectiveness_value > a2c_effectiveness_value:

    print(
        "PPO achieved better simulated treatment effectiveness."
    )

else:
```

```
print(
    "A2C achieved better simulated treatment effectiveness."
)

if ppo_stability_value < a2c_stability_value:

    print(
        "PPO showed more stable learning."
    )

else:

    print(
        "A2C showed more stable learning."
    )

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    a2c_rewards,
    label="A2C"
)

plt.plot(
    ppo_rewards,
    label="PPO"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Cumulative Reward"
)

plt.title(
    "A2C vs PPO Reward"
)

plt.legend()

plt.grid()

plt.show()

plt.figure()
```

```
        figsize=(10, 5)
    )

    plt.plot(
        a2c_effectiveness,
        label="A2C"
    )

    plt.plot(
        ppo_effectiveness,
        label="PPO"
    )

    plt.xlabel(
        "Episode"
    )

    plt.ylabel(
        "Simulated Effectiveness"
    )

    plt.title(
        "Treatment Effectiveness Comparison"
    )

    plt.legend()

    plt.grid()

    plt.show()


plt.figure(
    figsize=(10, 5)
)

plt.plot(
    a2c_stability,
    label="A2C"
)

plt.plot(
    ppo_stability,
    label="PPO"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Policy Loss Magnitude"
)
```

```

plt.title(
    "Learning Stability Comparison"
)

plt.legend()

plt.grid()

plt.show()

print("\n=====")
print("SIMULATION COMPLETED")
print("=====")

```

```

=====
ADAPTIVE PATIENT TREATMENT SIMULATION
A2C vs PPO
=====

```

```

Training A2C...
A2C Training Completed.

```

```

Training PPO...
PPO Training Completed.

```

```

=====
FINAL RESULTS
=====

```

```

A2C
Average Cumulative Reward: 93.485
Treatment Effectiveness: 1.0
Reward Standard Deviation: 7.349

```

```

PPO
Average Cumulative Reward: 96.744
Treatment Effectiveness: 1.0
Reward Standard Deviation: 7.038

```

```

=====
COMPARISON
=====

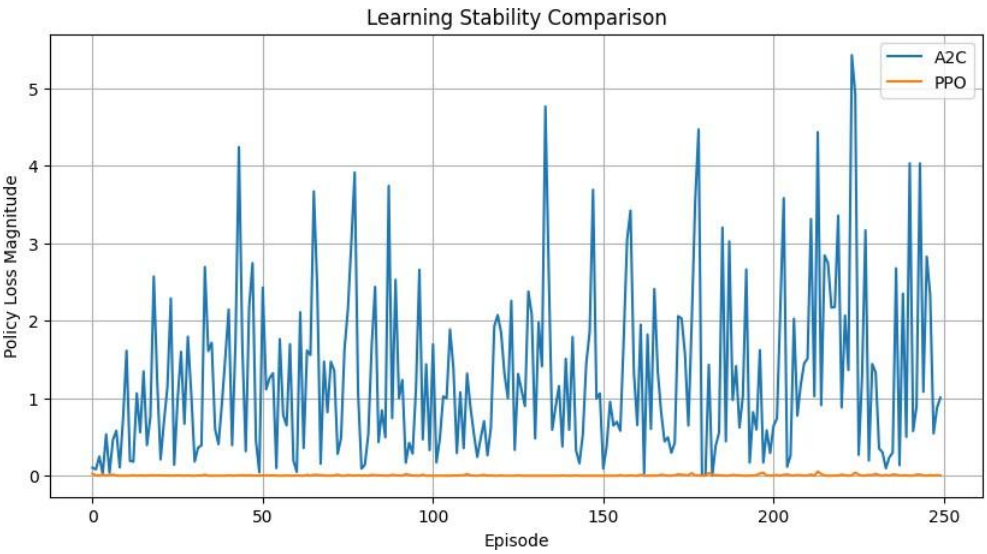
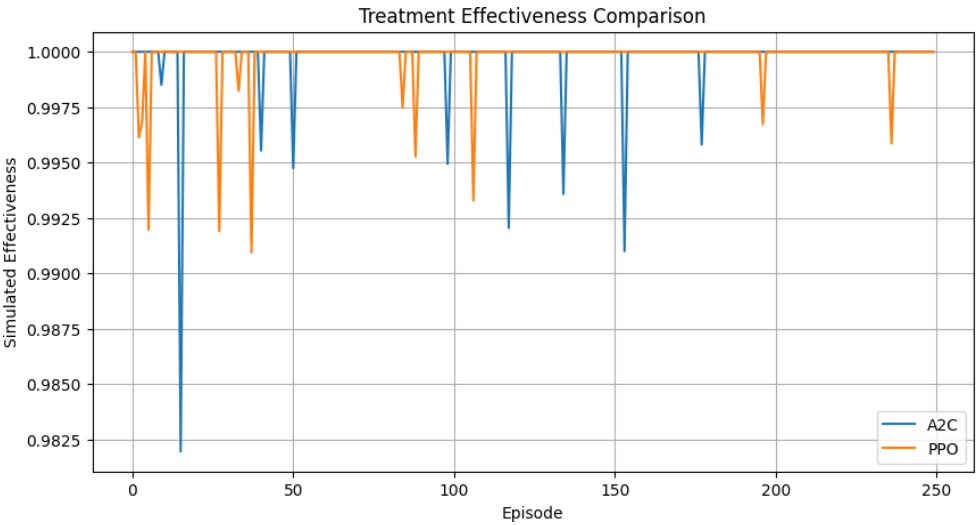
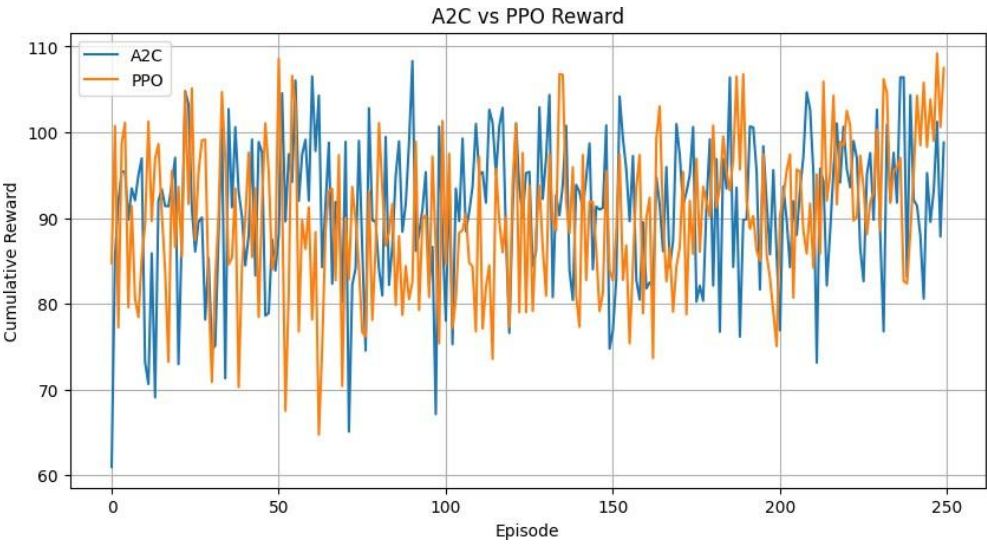
```

Metric	A2C	PPO
Cumulative Reward	93.485	96.744
Treatment Effectiveness	1.0	1.0
Reward Std. Deviation	7.349	7.038

```

PPO achieved higher cumulative reward.
A2C achieved better simulated treatment effectiveness.
PPO showed more stable learning.

```



=====
SIMULATION COMPLETED
=====

7. Develop a Deep Reinforcement Learning model using DDPG or PPO for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.

```
In [13]: import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

EPISODES = 300
MAX_STEPS = 80

GAMMA = 0.99
LEARNING_RATE = 0.001
CLIP_EPSILON = 0.2

GRID_SIZE = 10
OBSTACLES = 3

class DroneEnvironment:

    def __init__(self):

        self.size = GRID_SIZE

        self.reset()

    def reset(self):

        self.drone = [0, 0]

        self.goal = [
            self.size - 1,
            self.size - 1
        ]

        self.obstacles = []

        while len(self.obstacles) < OBSTACLES:

            obstacle = [
                random.randint(1, self.size - 2),
                random.randint(1, self.size - 2)
            ]

            if (
                obstacle != self.drone
                and
                obstacle != self.goal
                and
```

```
        obstacle not in self.obstacles
    ):

        self.obstacles.append(
            obstacle
        )

    return self.get_state()

def get_state(self):

    state = [
        self.drone[0] / self.size,
        self.drone[1] / self.size,

        self.goal[0] / self.size,
        self.goal[1] / self.size
    ]

    for obstacle in self.obstacles:

        state.append(
            obstacle[0] / self.size
        )

        state.append(
            obstacle[1] / self.size
        )

    return np.array(
        state,
        dtype=np.float32
    )

def distance_to_goal(self):

    return np.sqrt(
        (self.drone[0] - self.goal[0]) ** 2
        +
        (self.drone[1] - self.goal[1]) ** 2
    )

def step(self, action):

    old_distance = self.distance_to_goal()

    if action == 0:

        self.drone[0] += 1

    elif action == 1:

        self.drone[0] -= 1
```

```
elif action == 2:

    self.drone[1] -= 1

elif action == 3:

    self.drone[1] += 1

reward = -1

done = False

if (
    self.drone[0] < 0
    or
    self.drone[0] >= self.size
    or
    self.drone[1] < 0
    or
    self.drone[1] >= self.size
):

    reward = -20

    self.drone[0] = max(
        0,
        min(
            self.size - 1,
            self.drone[0]
        )
    )

    self.drone[1] = max(
        0,
        min(
            self.size - 1,
            self.drone[1]
        )
    )

elif self.drone in self.obstacles:

    reward = -100

    done = True

else:

    new_distance = self.distance_to_goal()

    if new_distance < old_distance:
```

```
        reward += 2

    else:

        reward -= 2

    if self.drone == self.goal:

        reward = 100

        done = True

    next_state = self.get_state()

    return (
        next_state,
        reward,
        done
    )

STATE_SIZE = 10
ACTION_SIZE = 4

class Actor(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(
                STATE_SIZE,
                128
            ),

            nn.ReLU(),

            nn.Linear(
                128,
                128
            ),

            nn.ReLU(),

            nn.Linear(
                128,
                ACTION_SIZE
            ),

            nn.Softmax(
```

```

        dim=-1
    )
)

def forward(self, state):

    return self.network(state)

class Critic(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(
                STATE_SIZE,
                128
            ),

            nn.ReLU(),

            nn.Linear(
                128,
                128
            ),

            nn.ReLU(),

            nn.Linear(
                128,
                1
            )
        )

    def forward(self, state):

        return self.network(state)

def calculate_returns(rewards):

    returns = []

    G = 0

    for reward in reversed(
        rewards
    ):

        G = reward + GAMMA * G

```

```
        returns.insert(
            0,
            G
        )

    return torch.FloatTensor(
        returns
    )

def train_ppo():

    env = DroneEnvironment()

    actor = Actor()

    critic = Critic()

    actor_optimizer = optim.Adam(
        actor.parameters(),
        lr=LEARNING_RATE
    )

    critic_optimizer = optim.Adam(
        critic.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []

    accuracy_history = []

    energy_history = []

    success_history = []

    for episode in range(
        EPISODES
    ):

        state = env.reset()

        states = []

        actions = []

        old_log_probs = []

        rewards = []

        energy = 0
```

```
success = 0

for step in range(
    MAX_STEPS
):

    state_tensor = torch.FloatTensor(
        state
    ).unsqueeze(0)

    probabilities = actor(
        state_tensor
    )

    distribution = torch.distributions.Categorical(
        probabilities
    )

    action = distribution.sample()

    log_prob = distribution.log_prob(
        action
    )

    next_state, reward, done = env.step(
        action.item()
    )

    states.append(
        state
    )

    actions.append(
        action.item()
    )

    old_log_probs.append(
        log_prob.item()
    )

    rewards.append(
        reward
    )

    energy += 1

    state = next_state

    if done:

        if env.drone == env.goal:

            success = 1
```

```
        break

states_tensor = torch.FloatTensor(
    np.array(states)
)

actions_tensor = torch.LongTensor(
    actions
)

old_log_probs_tensor = torch.FloatTensor(
    old_log_probs
)

returns = calculate_returns(
    rewards
)

values = critic(
    states_tensor
).squeeze()

advantages = (
    returns -
    values.detach()
)

if len(advantages) > 1:

    advantages = (
        advantages -
        advantages.mean()
    ) / (
        advantages.std()
        + 1e-8
    )

for _ in range(4):

    probabilities = actor(
        states_tensor
    )

    distribution = torch.distributions.Categorical(
        probabilities
    )

    new_log_probs = distribution.log_prob(
        actions_tensor
    )

    ratio = torch.exp(
```

```
        new_log_probs -
        old_log_probs_tensor
    )

    clipped_ratio = torch.clamp(
        ratio,
        1 - CLIP_EPSILON,
        1 + CLIP_EPSILON
    )

    actor_loss = -torch.min(
        ratio * advantages,
        clipped_ratio * advantages
    ).mean()

    actor_optimizer.zero_grad()

    actor_loss.backward()

    actor_optimizer.step()

    new_values = critic(
        states_tensor
    ).squeeze()

    critic_loss = nn.functional.mse_loss(
        new_values,
        returns
    )

    critic_optimizer.zero_grad()

    critic_loss.backward()

    critic_optimizer.step()

    final_distance = env.distance_to_goal()

    accuracy = 1 / (
        1 + final_distance
    )

    reward_history.append(
        sum(rewards)
    )

    accuracy_history.append(
        accuracy
    )

    energy_history.append(
        energy
    )
```

```
        success_history.append(
            success
        )

    return (
        actor,
        reward_history,
        accuracy_history,
        energy_history,
        success_history
    )

print("=====")
print("AUTONOMOUS DRONE NAVIGATION")
print("PPO DEEP REINFORCEMENT LEARNING")
print("=====")

print("\nTraining PPO...")

(
    trained_actor,
    rewards,
    accuracy,
    energy,
    success
) = train_ppo()

print("Training Completed.")

window = 30

average_reward = np.mean(
    rewards[-window:]
)

average_accuracy = np.mean(
    accuracy[-window:]
)

average_energy = np.mean(
    energy[-window:]
)

success_rate = (
    np.mean(
        success[-window:]
    ) * 100
)
```

```
print("\n=====")
print("FINAL EVALUATION")
print("=====")

print(
    "Average Reward:",
    round(
        average_reward,
        3
    )
)

print(
    "Navigation Accuracy:",
    round(
        average_accuracy,
        3
    )
)

print(
    "Average Energy Usage:",
    round(
        average_energy,
        2
    )
)

print(
    "Obstacle Avoidance / Success Rate:",
    round(
        success_rate,
        2
    ),
    "%"
)

print("\n=====")
print("POLICY CONVERGENCE")
print("=====")

first_average = np.mean(
    rewards[:30]
)

last_average = np.mean(
    rewards[-30:]
)

print(
    "Initial Average Reward:",
```

```
        round(
            first_average,
            3
        )
    )

print(
    "Final Average Reward:",
    round(
        last_average,
        3
    )
)

if last_average > first_average:

    print(
        "Policy shows learning improvement."
    )

else:

    print(
        "More training may be required."
    )

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    rewards
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Reward"
)

plt.title(
    "PPO Training Convergence"
)

plt.grid()

plt.show()

plt.figure()
```

```
        figsize=(10, 5)
    )

    plt.plot(
        accuracy
    )

    plt.xlabel(
        "Episode"
    )

    plt.ylabel(
        "Navigation Accuracy"
    )

    plt.title(
        "Drone Navigation Accuracy"
    )

    plt.grid()

    plt.show()

    plt.figure(
        figsize=(10, 5)
    )

    plt.plot(
        energy
    )

    plt.xlabel(
        "Episode"
    )

    plt.ylabel(
        "Energy Usage"
    )

    plt.title(
        "Drone Energy Consumption"
    )

    plt.grid()

    plt.show()

    plt.figure(
        figsize=(10, 5)
    )
```

```

plt.plot(
    np.cumsum(success)
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Successful Navigations"
)

plt.title(
    "Obstacle Avoidance and Navigation Success"
)

plt.grid()

plt.show()

print("\n=====")
print("PROJECT COMPLETED")
print("=====")

```

```

=====
AUTONOMOUS DRONE NAVIGATION
PPO DEEP REINFORCEMENT LEARNING
=====

```

```

Training PPO...
Training Completed.

```

```

=====
FINAL EVALUATION
=====

```

```

Average Reward: -64.1
Navigation Accuracy: 0.452
Average Energy Usage: 15.07
Obstacle Avoidance / Success Rate: 36.67 %

```

```

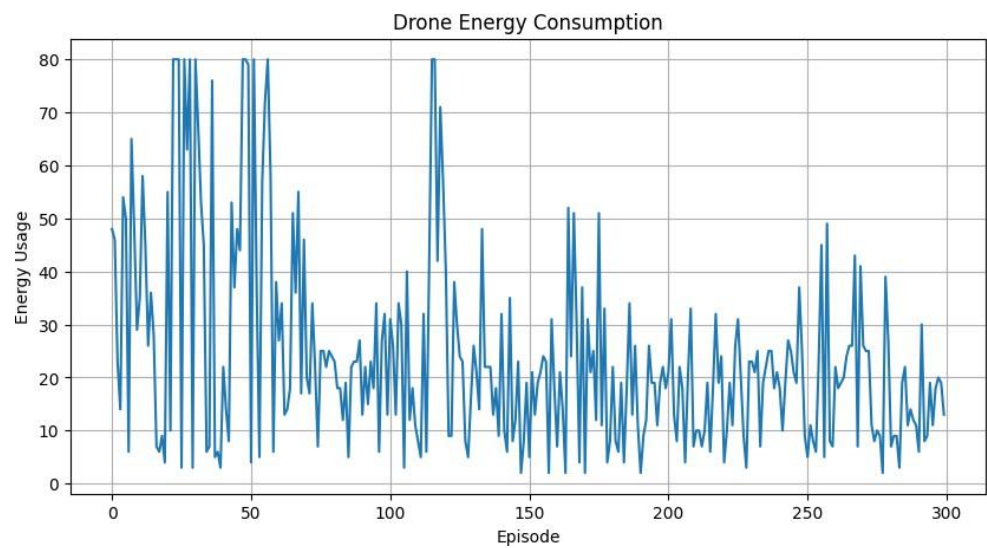
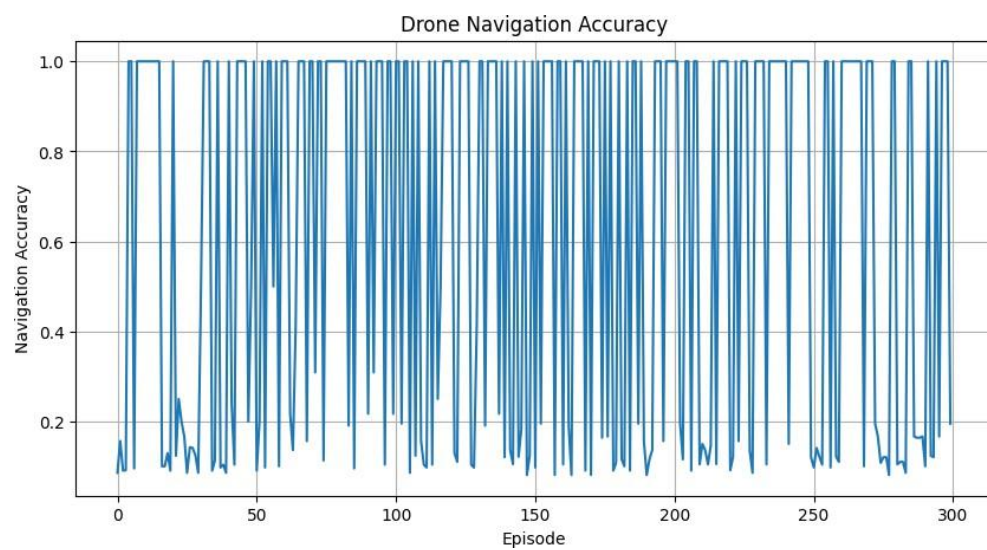
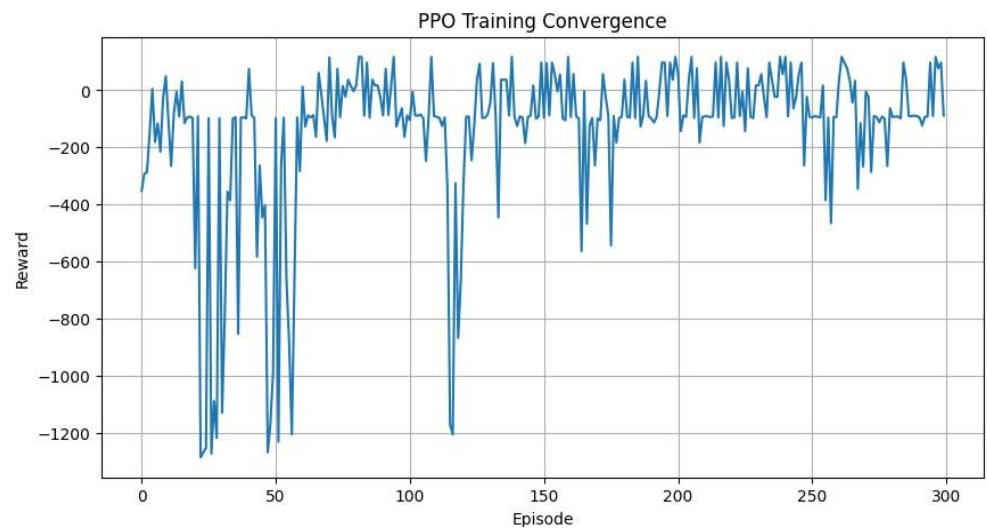
=====
POLICY CONVERGENCE
=====

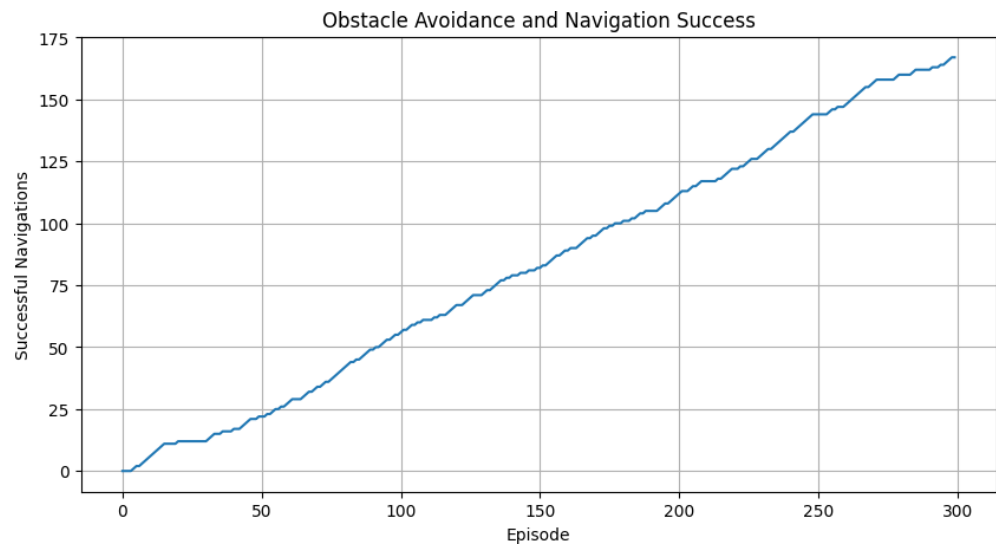
```

```

Initial Average Reward: -359.033
Final Average Reward: -64.1
Policy shows learning improvement.

```





=====
PROJECT COMPLETED
=====

8. Design and implement an A3C-based Reinforcement Learning framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.

```
In [14]: import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

EPISODES = 200
STEPS = 30

GAMMA = 0.95
LEARNING_RATE = 0.001

class CloudEnvironment:

    def __init__(self):
        self.reset()

    def reset(self):

        self.workload = random.uniform(
            0.2,
            0.9
        )

        self.cpu = random.uniform(
            0.3,
            0.7
        )

        self.resources = random.uniform(
            0.3,
            0.7
        )

        return self.get_state()

    def get_state(self):

        return np.array([
            self.workload,
            self.cpu,
            self.resources
        ], dtype=np.float32)

    def step(self, action):

        if action == 0:
```

```
        self.resources -= 0.1

    elif action == 1:
        self.resources = self.resources

    elif action == 2:
        self.resources += 0.1

    self.resources = np.clip(
        self.resources,
        0.1,
        1.0
    )

    self.cpu = (
        self.workload /
        self.resources
    )

    self.cpu = np.clip(
        self.cpu,
        0,
        1
    )

    response_time = (
        1 +
        self.workload /
        self.resources
    )

    resource_waste = abs(
        self.resources -
        self.workload
    )

    reward = (
        -response_time
        - resource_waste
    )

    self.workload += random.uniform(
        -0.1,
        0.1
    )

    self.workload = np.clip(
        self.workload,
        0.1,
        1.0
    )

    next_state = self.get_state()
```

```
        return (
            next_state,
            reward,
            response_time,
            self.cpu,
            self.resources
        )

class PolicyNetwork(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(
                3,
                64
            ),

            nn.ReLU(),

            nn.Linear(
                64,
                64
            ),

            nn.ReLU(),

            nn.Linear(
                64,
                3
            ),

            nn.Softmax(
                dim=-1
            )
        )

    def forward(self, state):

        return self.network(state)

class ValueNetwork(nn.Module):

    def __init__(self):

        super().__init__()
```

```
self.network = nn.Sequential(

    nn.Linear(
        3,
        64
    ),

    nn.ReLU(),

    nn.Linear(
        64,
        64
    ),

    nn.ReLU(),

    nn.Linear(
        64,
        1
    )
)

def forward(self, state):

    return self.network(state)

def calculate_returns(rewards):

    returns = []

    total = 0

    for reward in reversed(
        rewards
    ):

        total = (
            reward +
            GAMMA * total
        )

        returns.insert(
            0,
            total
        )

    return torch.FloatTensor(
        returns
    )

def train_vanilla_pg():
```

```
env = CloudEnvironment()

policy = PolicyNetwork()

optimizer = optim.Adam(
    policy.parameters(),
    lr=LEARNING_RATE
)

reward_history = []

response_history = []

cpu_history = []

efficiency_history = []

for episode in range(
    EPISODES
):
    state = env.reset()

    log_probs = []

    rewards = []

    responses = []

    cpus = []

    resources = []

    for step in range(
        STEPS
    ):
        state_tensor = torch.FloatTensor(
            state
        )

        probabilities = policy(
            state_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        action = distribution.sample()

        log_prob = distribution.log_prob(
```

```
        action
    )

    (
        next_state,
        reward,
        response,
        cpu,
        resource
    ) = env.step(
        action.item()
    )

    log_probs.append(
        log_prob
    )

    rewards.append(
        reward
    )

    responses.append(
        response
    )

    cpus.append(
        cpu
    )

    resources.append(
        resource
    )

    state = next_state

returns = calculate_returns(
    rewards
)

if len(returns) > 1:

    returns = (
        returns -
        returns.mean()
    ) / (
        returns.std()
        + 1e-8
    )

loss = 0

for log_prob, G in zip(
    log_probs,
```

```

        returns
    ):

        loss += (
            -log_prob * G
        )

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    reward_history.append(
        sum(rewards)
    )

    response_history.append(
        np.mean(responses)
    )

    cpu_history.append(
        np.mean(cpus) * 100
    )

    efficiency_history.append(
        np.mean(
            np.array(cpus) /
            (np.array(resources) + 1e-8)
        )
    )

    return (
        policy,
        reward_history,
        response_history,
        cpu_history,
        efficiency_history
    )

def train_a3c():

    global_policy = PolicyNetwork()

    optimizer = optim.Adam(
        global_policy.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []

    response_history = []

```

```
cpu_history = []

efficiency_history = []

workers = 4

episodes_per_worker = (
    EPISODES // workers
)

for worker in range(
    workers
):

    env = CloudEnvironment()

    for episode in range(
        episodes_per_worker
    ):

        state = env.reset()

        log_probs = []

        rewards = []

        responses = []

        cpus = []

        resources = []

        for step in range(
            STEPS
        ):

            state_tensor = torch.FloatTensor(
                state
            )

            probabilities = global_policy(
                state_tensor
            )

            distribution = torch.distributions.Categorical(
                probabilities
            )

            action = distribution.sample()

            log_prob = distribution.log_prob(
                action
```

```
)

(
    next_state,
    reward,
    response,
    cpu,
    resource
) = env.step(
    action.item()
)

log_probs.append(
    log_prob
)

rewards.append(
    reward
)

responses.append(
    response
)

cpus.append(
    cpu
)

resources.append(
    resource
)

state = next_state

returns = calculate_returns(
    rewards
)

if len(returns) > 1:

    returns = (
        returns -
        returns.mean()
    ) / (
        returns.std()
        + 1e-8
    )

loss = 0

for log_prob, G in zip(
    log_probs,
    returns
```

```

):

    loss += (
        -log_prob * G
    )

optimizer.zero_grad()

loss.backward()

optimizer.step()

reward_history.append(
    sum(rewards)
)

response_history.append(
    np.mean(responses)
)

cpu_history.append(
    np.mean(cpus) * 100
)

efficiency_history.append(
    np.mean(
        np.array(cpus) /
        (np.array(resources) + 1e-8)
    )
)

return (
    global_policy,
    reward_history,
    response_history,
    cpu_history,
    efficiency_history
)

print("=====")
print("CLOUD RESOURCE ALLOCATION")
print("A3C vs VANILLA POLICY GRADIENT")
print("=====")

print("\nTraining Vanilla Policy Gradient...")

(
    pg_policy,
    pg_rewards,
    pg_response,
    pg_cpu,
    pg_efficiency

```

```
) = train_vanilla_pg()

print(
    "Vanilla Policy Gradient completed."
)

print("\nTraining A3C...")

(
    a3c_policy,
    a3c_rewards,
    a3c_response,
    a3c_cpu,
    a3c_efficiency
) = train_a3c()

print(
    "A3C completed."
)

window = 30

pg_reward = np.mean(
    pg_rewards[-window:]
)

a3c_reward = np.mean(
    a3c_rewards[-window:]
)

pg_response_value = np.mean(
    pg_response[-window:]
)

a3c_response_value = np.mean(
    a3c_response[-window:]
)

pg_cpu_value = np.mean(
    pg_cpu[-window:]
)

a3c_cpu_value = np.mean(
    a3c_cpu[-window:]
)

pg_efficiency_value = np.mean(
    pg_efficiency[-window:]
)

a3c_efficiency_value = np.mean(
```

```
        a3c_efficiency[-window:]
    )

    print("\n=====")
    print("FINAL PERFORMANCE")
    print("=====")

    print("\nVanilla Policy Gradient")

    print(
        "Average Reward:",
        round(
            pg_reward,
            3
        )
    )

    print(
        "Response Time:",
        round(
            pg_response_value,
            3
        )
    )

    print(
        "CPU Utilization:",
        round(
            pg_cpu_value,
            2
        ),
        "%"
    )

    print(
        "Resource Efficiency:",
        round(
            pg_efficiency_value,
            3
        )
    )

    print("\nA3C")

    print(
        "Average Reward:",
        round(
            a3c_reward,
            3
        )
    )
)
```

```

print(
    "Response Time:",
    round(
        a3c_response_value,
        3
    )
)

print(
    "CPU Utilization:",
    round(
        a3c_cpu_value,
        2
    ),
    "%"
)

print(
    "Resource Efficiency:",
    round(
        a3c_efficiency_value,
        3
    )
)

print("\n=====")
print("COMPARISON")
print("=====")

print(
    "\nMetric".ljust(25),
    "Vanilla PG".ljust(15),
    "A3C"
)

print(
    "Average Reward".ljust(25),
    round(pg_reward, 3),
    " " * 8,
    round(a3c_reward, 3)
)

print(
    "Response Time".ljust(25),
    round(pg_response_value, 3),
    " " * 8,
    round(a3c_response_value, 3)
)

print(
    "CPU Utilization (%)".ljust(25),

```

```
round(pg_cpu_value, 2),
" " * 8,
round(a3c_cpu_value, 2)
)

print(
    "Resource Efficiency".ljust(25),
    round(pg_efficiency_value, 3),
    " " * 8,
    round(a3c_efficiency_value, 3)
)

if a3c_response_value < pg_response_value:

    print(
        "\nA3C achieved lower response time."
    )

else:

    print(
        "\nVanilla Policy Gradient achieved lower response time."
    )

if a3c_efficiency_value > pg_efficiency_value:

    print(
        "A3C achieved better resource efficiency."
    )

else:

    print(
        "Vanilla Policy Gradient achieved better resource efficiency."
    )

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    pg_rewards,
    label="Vanilla Policy Gradient"
)

plt.plot(
    a3c_rewards,
    label="A3C"
)
```

```
plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Reward"
)

plt.title(
    "Training Reward Comparison"
)

plt.legend()

plt.grid()

plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    pg_response,
    label="Vanilla Policy Gradient"
)

plt.plot(
    a3c_response,
    label="A3C"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Response Time"
)

plt.title(
    "Response Time Comparison"
)

plt.legend()

plt.grid()

plt.show()

plt.figure(
```

```
        figsize=(10, 5)
    )

    plt.plot(
        pg_cpu,
        label="Vanilla Policy Gradient"
    )

    plt.plot(
        a3c_cpu,
        label="A3C"
    )

    plt.xlabel(
        "Episode"
    )

    plt.ylabel(
        "CPU Utilization (%)"
    )

    plt.title(
        "CPU Utilization Comparison"
    )

    plt.legend()

    plt.grid()

    plt.show()


plt.figure(
    figsize=(10, 5)
)

plt.plot(
    pg_efficiency,
    label="Vanilla Policy Gradient"
)

plt.plot(
    a3c_efficiency,
    label="A3C"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Resource Efficiency"
)
```

```
plt.title(
    "Resource Efficiency Comparison"
)

plt.legend()

plt.grid()

plt.show()

print("\n=====")
print("PROJECT COMPLETED")
print("=====")

=====
CLOUD RESOURCE ALLOCATION
A3C vs VANILLA POLICY GRADIENT
=====

Training Vanilla Policy Gradient...
Vanilla Policy Gradient completed.

Training A3C...
A3C completed.

=====
FINAL PERFORMANCE
=====

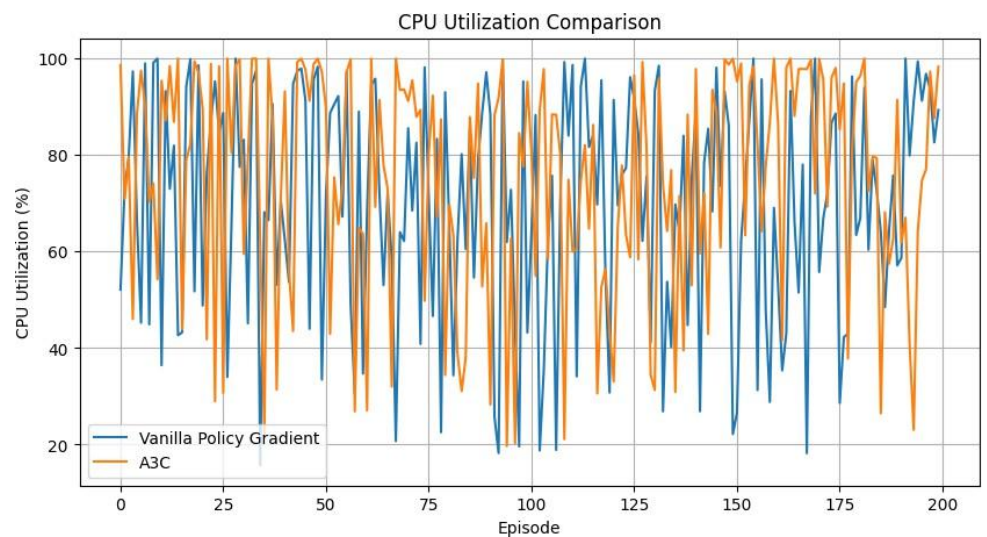
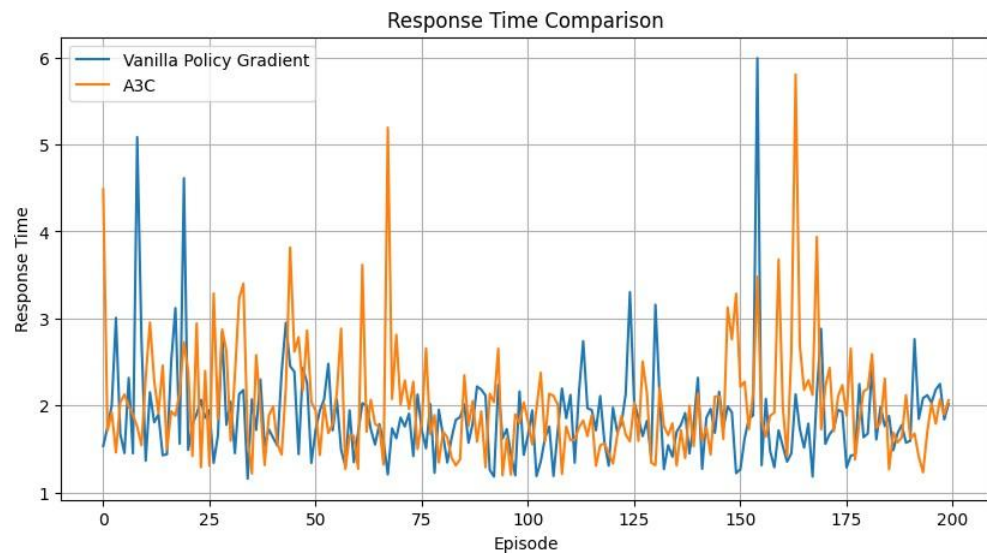
Vanilla Policy Gradient
Average Reward: -63.773
Response Time: 1.848
CPU Utilization: 73.65 %
Resource Efficiency: 1.112

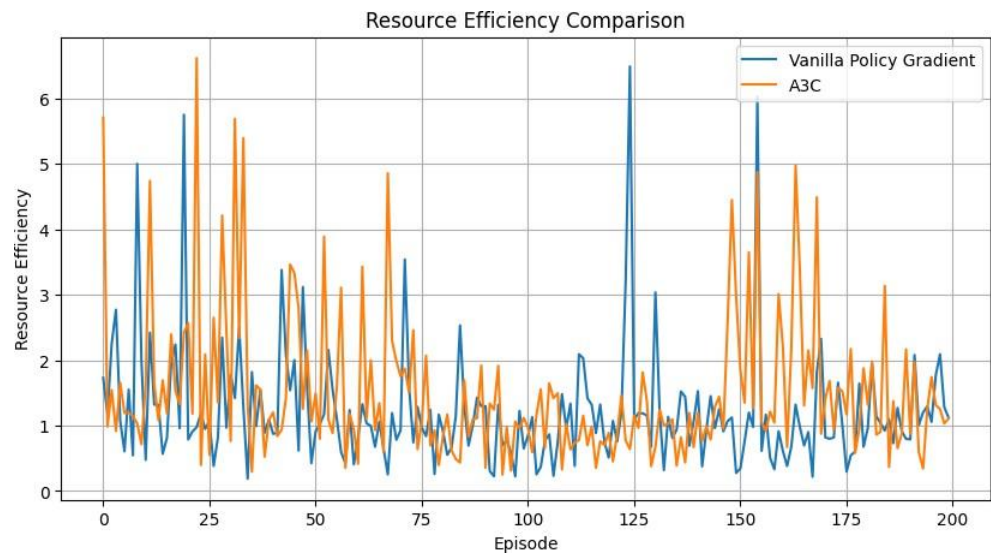
A3C
Average Reward: -64.315
Response Time: 1.899
CPU Utilization: 75.99 %
Resource Efficiency: 1.299

=====
COMPARISON
=====

Metric                Vanilla PG    A3C
Average Reward        -63.773      -64.315
Response Time         1.848        1.899
CPU Utilization (%)   73.65        75.99
Resource Efficiency    1.112        1.299
```

Vanilla Policy Gradient achieved lower response time.
A3C achieved better resource efficiency.





=====

PROJECT COMPLETED

=====

9. Develop a Policy Gradient-based predictive maintenance system for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of REINFORCE and PPO.

```
In [15]: import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

EPISODES = 250
STEPS = 40

GAMMA = 0.95
LEARNING_RATE = 0.001
CLIP_EPSILON = 0.2

class MachineEnvironment:

    def __init__(self):
        self.reset()

    def reset(self):

        self.health = random.uniform(
            0.5,
            1.0
        )

        self.temperature = random.uniform(
            0.2,
            0.6
        )

        self.vibration = random.uniform(
            0.1,
            0.5
        )

        self.age = random.uniform(
            0.1,
            0.5
        )

        return self.get_state()

    def get_state(self):

        return np.array([
            self.health,
            self.temperature,
```

```
        self.vibration,
        self.age
    ], dtype=np.float32)

def step(self, action):

    maintenance_cost = 0
    downtime = 0

    if action == 0:

        maintenance_cost = 0

        self.health -= random.uniform(
            0.01,
            0.05
        )

        self.temperature += random.uniform(
            0.00,
            0.03
        )

        self.vibration += random.uniform(
            0.00,
            0.03
        )

    elif action == 1:

        maintenance_cost = 2

        self.health += 0.12

        self.temperature -= 0.05

        self.vibration -= 0.05

        downtime = 1

    elif action == 2:

        maintenance_cost = 4

        self.health += 0.20

        self.temperature -= 0.10

        self.vibration -= 0.10

        downtime = 2

    self.health = np.clip(
```

```
        self.health,
        0,
        1
    )

    self.temperature = np.clip(
        self.temperature,
        0,
        1
    )

    self.vibration = np.clip(
        self.vibration,
        0,
        1
    )

    self.age += 0.02

    self.age = np.clip(
        self.age,
        0,
        1
    )

    failure = False

    if self.health < 0.20:

        failure = True

        downtime = 5

        failure_cost = 10

    else:

        failure_cost = 0

    operating_reward = 3 * self.health

    reward = (
        operating_reward
        - maintenance_cost
        - downtime
        - failure_cost
    )

    done = failure

    return (
        self.get_state(),
        reward,
```

```
        done,  
        maintenance_cost,  
        downtime,  
        failure  
    )
```

```
class PolicyNetwork(nn.Module):  
  
    def __init__(self):  
  
        super().__init__()  
  
        self.network = nn.Sequential(  
  
            nn.Linear(  
                4,  
                64  
            ),  
  
            nn.ReLU(),  
  
            nn.Linear(  
                64,  
                64  
            ),  
  
            nn.ReLU(),  
  
            nn.Linear(  
                64,  
                3  
            ),  
  
            nn.Softmax(  
                dim=-1  
            )  
        )  
  
    def forward(self, state):  
  
        return self.network(state)
```

```
class ValueNetwork(nn.Module):  
  
    def __init__(self):  
  
        super().__init__()  
  
        self.network = nn.Sequential(  
  
            nn.Linear(  
                4,  
                64  
            ),  
  
            nn.ReLU(),  
  
            nn.Linear(  
                64,  
                64  
            ),  
  
            nn.ReLU(),  
  
            nn.Linear(  
                64,  
                1  
            )  
        )  
  
    def forward(self, state):  
  
        return self.network(state)
```

```
        4,
        64
    ),

    nn.ReLU(),

    nn.Linear(
        64,
        64
    ),

    nn.ReLU(),

    nn.Linear(
        64,
        1
    )
)

def forward(self, state):

    return self.network(state)

def calculate_returns(rewards):

    returns = []

    total = 0

    for reward in reversed(
        rewards
    ):

        total = (
            reward
            +
            GAMMA * total
        )

        returns.insert(
            0,
            total
        )

    return torch.FloatTensor(
        returns
    )

def train_reinforce():

    env = MachineEnvironment()
```

```
policy = PolicyNetwork()

optimizer = optim.Adam(
    policy.parameters(),
    lr=LEARNING_RATE
)

reward_history = []
cost_history = []
downtime_history = []
failure_history = []

for episode in range(
    EPISODES
):

    state = env.reset()

    log_probs = []

    rewards = []

    total_cost = 0

    total_downtime = 0

    failure = 0

    for step in range(
        STEPS
    ):

        state_tensor = torch.FloatTensor(
            state
        )

        probabilities = policy(
            state_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        action = distribution.sample()

        log_prob = distribution.log_prob(
            action
        )

        (
            next_state,
```

```
        reward,
        done,
        cost,
        downtime,
        failed
    ) = env.step(
        action.item()
    )

    log_probs.append(
        log_prob
    )

    rewards.append(
        reward
    )

    total_cost += cost

    total_downtime += downtime

    if failed:

        failure = 1

    state = next_state

    if done:

        break

    returns = calculate_returns(
        rewards
    )

    if len(returns) > 1:

        returns = (
            returns
            -
            returns.mean()
        ) / (
            returns.std()
            +
            1e-8
        )

    loss = 0

    for log_prob, G in zip(
        log_probs,
        returns
    ):

```

```
        loss += (
            -log_prob * G
        )

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    reward_history.append(
        sum(rewards)
    )

    cost_history.append(
        total_cost
    )

    downtime_history.append(
        total_downtime
    )

    failure_history.append(
        failure
    )

    return (
        policy,
        reward_history,
        cost_history,
        downtime_history,
        failure_history
    )

def train_ppo():

    env = MachineEnvironment()

    policy = PolicyNetwork()

    value = ValueNetwork()

    policy_optimizer = optim.Adam(
        policy.parameters(),
        lr=LEARNING_RATE
    )

    value_optimizer = optim.Adam(
        value.parameters(),
        lr=LEARNING_RATE
    )
```

```
reward_history = []
cost_history = []
downtime_history = []
failure_history = []

for episode in range(
    EPISODES
):

    state = env.reset()

    states = []

    actions = []

    old_log_probs = []

    rewards = []

    total_cost = 0

    total_downtime = 0

    failure = 0

    for step in range(
        STEPS
    ):

        state_tensor = torch.FloatTensor(
            state
        ).unsqueeze(0)

        probabilities = policy(
            state_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        action = distribution.sample()

        log_prob = distribution.log_prob(
            action
        )

        (
            next_state,
            reward,
            done,
            cost,
```

```
        downtime,
        failed
    ) = env.step(
        action.item()
    )

    states.append(
        state
    )

    actions.append(
        action.item()
    )

    old_log_probs.append(
        log_prob.item()
    )

    rewards.append(
        reward
    )

    total_cost += cost

    total_downtime += downtime

    if failed:

        failure = 1

    state = next_state

    if done:

        break

    states_tensor = torch.FloatTensor(
        np.array(states)
    )

    actions_tensor = torch.LongTensor(
        actions
    )

    old_log_probs_tensor = torch.FloatTensor(
        old_log_probs
    )

    returns = calculate_returns(
        rewards
    )

    values = value(
```

```
        states_tensor
    ).squeeze()

    advantages = (
        returns
        -
        values.detach()
    )

    if len(advantages) > 1:

        advantages = (
            advantages
            -
            advantages.mean()
        ) / (
            advantages.std()
            +
            1e-8
        )

    for _ in range(4):

        probabilities = policy(
            states_tensor
        )

        distribution = torch.distributions.Categorical(
            probabilities
        )

        new_log_probs = distribution.log_prob(
            actions_tensor
        )

        ratio = torch.exp(
            new_log_probs
            -
            old_log_probs_tensor
        )

        clipped_ratio = torch.clamp(
            ratio,
            1 - CLIP_EPSILON,
            1 + CLIP_EPSILON
        )

        actor_loss = -torch.min(
            ratio * advantages,
            clipped_ratio * advantages
        ).mean()

    policy_optimizer.zero_grad()
```

```
        actor_loss.backward()

        policy_optimizer.step()

        new_values = value(
            states_tensor
        ).squeeze()

        critic_loss = nn.functional.mse_loss(
            new_values,
            returns
        )

        value_optimizer.zero_grad()

        critic_loss.backward()

        value_optimizer.step()

    reward_history.append(
        sum(rewards)
    )

    cost_history.append(
        total_cost
    )

    downtime_history.append(
        total_downtime
    )

    failure_history.append(
        failure
    )

    return (
        policy,
        reward_history,
        cost_history,
        downtime_history,
        failure_history
    )

print("=====")
print("PREDICTIVE MAINTENANCE")
print("REINFORCE vs PPO")
print("=====")

print("\nTraining REINFORCE...")
```

```
(
    reinforce_policy,
    reinforce_rewards,
    reinforce_cost,
    reinforce_downtime,
    reinforce_failure
) = train_reinforce()

print(
    "REINFORCE Training Completed."
)

print("\nTraining PPO...")

(
    ppo_policy,
    ppo_rewards,
    ppo_cost,
    ppo_downtime,
    ppo_failure
) = train_ppo()

print(
    "PPO Training Completed."
)

window = 30

reinforce_reward = np.mean(
    reinforce_rewards[-window:]
)

ppo_reward = np.mean(
    ppo_rewards[-window:]
)

reinforce_cost_value = np.mean(
    reinforce_cost[-window:]
)

ppo_cost_value = np.mean(
    ppo_cost[-window:]
)

reinforce_downtime_value = np.mean(
    reinforce_downtime[-window:]
)

ppo_downtime_value = np.mean(
    ppo_downtime[-window:]
)
```

```
)

reinforce_failures = sum(
    reinforce_failure[-window:]
)

ppo_failures = sum(
    ppo_failure[-window:]
)

print("\n=====")
print("FINAL PERFORMANCE")
print("=====")

print("\nREINFORCE")

print(
    "Average Reward:",
    round(
        reinforce_reward,
        3
    )
)

print(
    "Maintenance Cost:",
    round(
        reinforce_cost_value,
        3
    )
)

print(
    "Downtime:",
    round(
        reinforce_downtime_value,
        3
    )
)

print(
    "Failures:",
    reinforce_failures
)

print("\nPPO")

print(
    "Average Reward:",
    round(
```

```
        ppo_reward,
        3
    )
)

print(
    "Maintenance Cost:",
    round(
        ppo_cost_value,
        3
    )
)

print(
    "Downtime:",
    round(
        ppo_downtime_value,
        3
    )
)

print(
    "Failures:",
    ppo_failures
)

print("\n=====")
print("COMPARISON")
print("=====")

print(
    "\nMetric".ljust(25),
    "REINFORCE".ljust(15),
    "PPO"
)

print(
    "Average Reward".ljust(25),
    round(
        reinforce_reward,
        3
    ),
    " " * 8,
    round(
        ppo_reward,
        3
    )
)
```

```
print(
    "Maintenance Cost".ljust(25),
    round(
        reinforce_cost_value,
        3
    ),
    " " * 8,
    round(
        ppo_cost_value,
        3
    )
)

print(
    "Downtime".ljust(25),
    round(
        reinforce_downtime_value,
        3
    ),
    " " * 8,
    round(
        ppo_downtime_value,
        3
    )
)

print(
    "Failures".ljust(25),
    reinforce_failures,
    " " * 8,
    ppo_failures
)

if ppo_cost_value < reinforce_cost_value:

    print(
        "\nPPO achieved lower maintenance cost."
    )

else:

    print(
        "\nREINFORCE achieved lower maintenance cost."
    )

if ppo_downtime_value < reinforce_downtime_value:

    print(
        "PPO achieved lower downtime."
```

```
)

else:

    print(
        "REINFORCE achieved lower downtime."
    )

if ppo_reward > reinforce_reward:

    print(
        "PPO achieved higher reward."
    )

else:

    print(
        "REINFORCE achieved higher reward."
    )

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    reinforce_rewards,
    label="REINFORCE"
)

plt.plot(
    ppo_rewards,
    label="PPO"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Reward"
)

plt.title(
    "REINFORCE vs PPO Training Reward"
)

plt.legend()

plt.grid()

plt.show()
```

```
plt.figure(
    figsize=(10, 5)
)

plt.plot(
    reinforce_cost,
    label="REINFORCE"
)

plt.plot(
    ppo_cost,
    label="PPO"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Maintenance Cost"
)

plt.title(
    "Maintenance Cost Comparison"
)

plt.legend()

plt.grid()

plt.show()


plt.figure(
    figsize=(10, 5)
)

plt.plot(
    reinforce_downtime,
    label="REINFORCE"
)

plt.plot(
    ppo_downtime,
    label="PPO"
)

plt.xlabel(
    "Episode"
)
```

```
plt.ylabel(
    "Downtime"
)

plt.title(
    "Machine Downtime Comparison"
)

plt.legend()

plt.grid()

plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    np.cumsum(
        reinforce_failure
    ),
    label="REINFORCE"
)

plt.plot(
    np.cumsum(
        ppo_failure
    ),
    label="PPO"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Cumulative Failures"
)

plt.title(
    "Failure Comparison"
)

plt.legend()

plt.grid()

plt.show()

print("\n=====")
```

```
print("PROJECT COMPLETED")
print("=====")
```

```
=====
PREDICTIVE MAINTENANCE
REINFORCE vs PPO
=====
```

```
Training REINFORCE...
REINFORCE Training Completed.
```

```
Training PPO...
PPO Training Completed.
```

```
=====
FINAL PERFORMANCE
=====
```

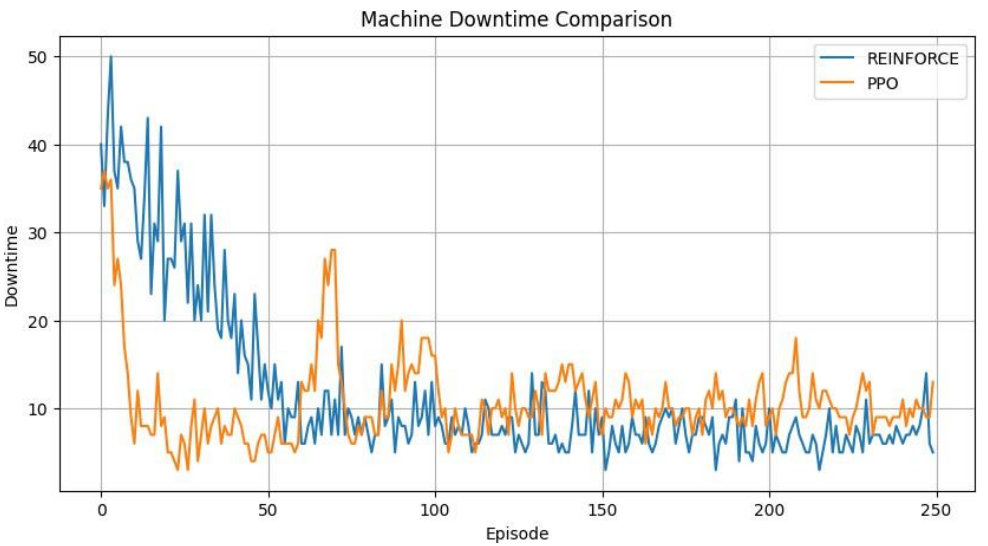
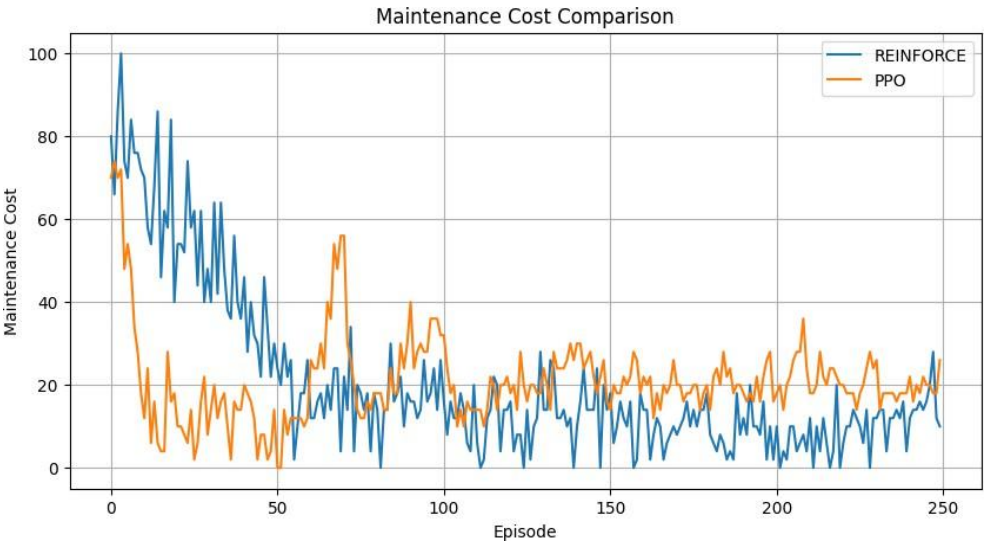
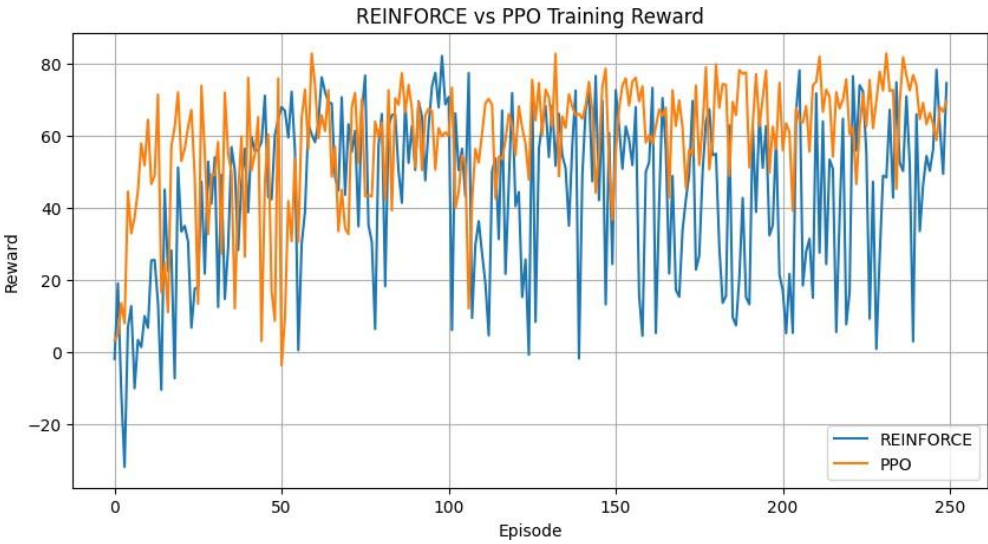
```
REINFORCE
Average Reward: 50.59
Maintenance Cost: 12.133
Downtime: 7.067
Failures: 6
```

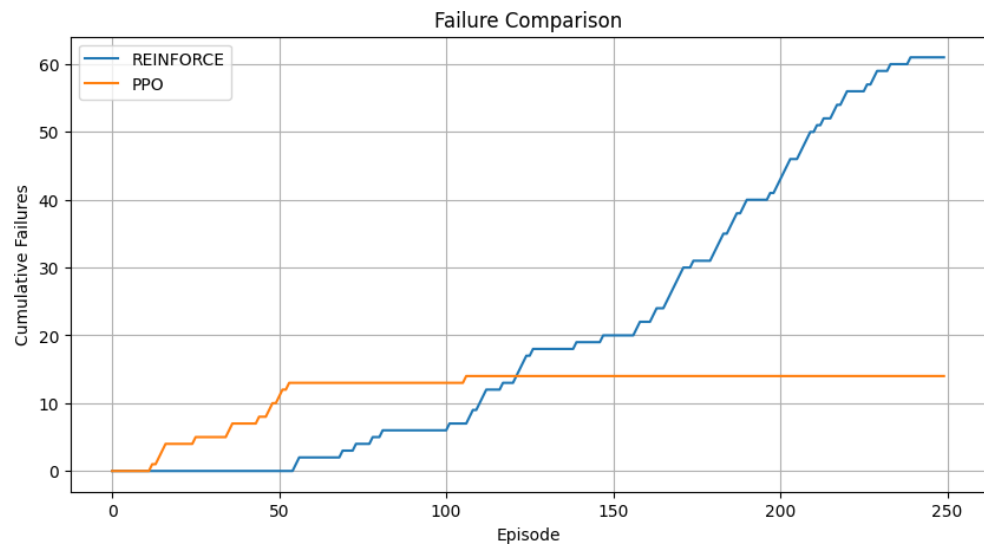
```
PPO
Average Reward: 67.891
Maintenance Cost: 19.467
Downtime: 9.733
Failures: 0
```

```
=====
COMPARISON
=====
```

Metric	REINFORCE	PPO
Average Reward	50.59	67.891
Maintenance Cost	12.133	19.467
Downtime	7.067	9.733
Failures	6	0

```
REINFORCE achieved lower maintenance cost.
REINFORCE achieved lower downtime.
PPO achieved higher reward.
```





```
=====
PROJECT COMPLETED
=====
```

10. Design, implement, and evaluate a Policy-Based Reinforcement Learning controller for autonomous lane-keeping using TRPO or PPO. Compare the selected algorithm with A2C based on lane deviation, safety, reward convergence, and training efficiency.

```
In [16]: import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

EPISODES = 250
STEPS = 50

GAMMA = 0.99
LEARNING_RATE = 0.001
CLIP_EPSILON = 0.2

class LaneEnvironment:

    def __init__(self):
        self.reset()

    def reset(self):

        self.position = random.uniform(
            -0.3,
            0.3
        )

        self.speed = random.uniform(
            0.4,
            0.8
        )

        self.curvature = random.uniform(
            -0.2,
            0.2
        )

        self.steering = 0

        return self.get_state()

    def get_state(self):

        return np.array([
            self.position,
            self.speed,
            self.curvature,
            self.steering
        ], dtype=np.float32)
```

```
def step(self, action):

    if action == 0:

        self.steering = -0.1

    elif action == 1:

        self.steering = 0

    elif action == 2:

        self.steering = 0.1

    self.position += (
        self.steering
        +
        self.curvature * 0.1
        +
        random.uniform(
            -0.02,
            0.02
        )
    )

    self.curvature += random.uniform(
        -0.03,
        0.03
    )

    self.curvature = np.clip(
        self.curvature,
        -0.5,
        0.5
    )

    deviation = abs(
        self.position
    )

    reward = 5

    reward -= (
        10 * deviation
    )

    if deviation < 0.1:

        reward += 3

    if deviation > 0.5:
```

```
        reward -= 30

        done = True

    else:

        done = False

    safety = 1

    if deviation > 0.4:

        safety = 0

    next_state = self.get_state()

    return (
        next_state,
        reward,
        done,
        deviation,
        safety
    )

class Actor(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(
                4,
                64
            ),

            nn.ReLU(),

            nn.Linear(
                64,
                64
            ),

            nn.ReLU(),

            nn.Linear(
                64,
                3
            ),

            nn.Softmax(
```

```

        dim=-1
    )
)

def forward(self, state):

    return self.network(state)

class Critic(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(
                4,
                64
            ),

            nn.ReLU(),

            nn.Linear(
                64,
                64
            ),

            nn.ReLU(),

            nn.Linear(
                64,
                1
            )
        )

    def forward(self, state):

        return self.network(state)

def calculate_returns(rewards):

    returns = []

    G = 0

    for reward in reversed(
        rewards
    ):

        G = (

```

```

        reward
        +
        GAMMA * G
    )

    returns.insert(
        0,
        G
    )

    return torch.FloatTensor(
        returns
    )

def train_ppo():

    env = LaneEnvironment()

    actor = Actor()

    critic = Critic()

    actor_optimizer = optim.Adam(
        actor.parameters(),
        lr=LEARNING_RATE
    )

    critic_optimizer = optim.Adam(
        critic.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []

    deviation_history = []

    safety_history = []

    for episode in range(
        EPISODES
    ):

        state = env.reset()

        states = []

        actions = []

        old_log_probs = []

        rewards = []

```

```
deviations = []

safety_values = []

for step in range(
    STEPS
):

    state_tensor = torch.FloatTensor(
        state
    ).unsqueeze(0)

    probabilities = actor(
        state_tensor
    )

    distribution = torch.distributions.Categorical(
        probabilities
    )

    action = distribution.sample()

    log_prob = distribution.log_prob(
        action
    )

    (
        next_state,
        reward,
        done,
        deviation,
        safety
    ) = env.step(
        action.item()
    )

    states.append(
        state
    )

    actions.append(
        action.item()
    )

    old_log_probs.append(
        log_prob.item()
    )

    rewards.append(
        reward
    )

    deviations.append(
```

```
        deviation
    )

    safety_values.append(
        safety
    )

    state = next_state

    if done:

        break

    states_tensor = torch.FloatTensor(
        np.array(states)
    )

    actions_tensor = torch.LongTensor(
        actions
    )

    old_log_probs_tensor = torch.FloatTensor(
        old_log_probs
    )

    returns = calculate_returns(
        rewards
    )

    values = critic(
        states_tensor
    ).squeeze()

    advantages = (
        returns
        -
        values.detach()
    )

    if len(advantages) > 1:

        advantages = (
            advantages
            -
            advantages.mean()
        ) / (
            advantages.std()
            +
            1e-8
        )

    for _ in range(4):
```

```
probabilities = actor(
    states_tensor
)

distribution = torch.distributions.Categorical(
    probabilities
)

new_log_probs = distribution.log_prob(
    actions_tensor
)

ratio = torch.exp(
    new_log_probs
    -
    old_log_probs_tensor
)

clipped_ratio = torch.clamp(
    ratio,
    1 - CLIP_EPSILON,
    1 + CLIP_EPSILON
)

actor_loss = -torch.min(
    ratio * advantages,
    clipped_ratio * advantages
).mean()

actor_optimizer.zero_grad()

actor_loss.backward()

actor_optimizer.step()

new_values = critic(
    states_tensor
).squeeze()

critic_loss = nn.functional.mse_loss(
    new_values,
    returns
)

critic_optimizer.zero_grad()

critic_loss.backward()

critic_optimizer.step()

reward_history.append(
    sum(rewards)
)
```

```
        deviation_history.append(
            np.mean(deviations)
        )

        safety_history.append(
            np.mean(safety_values)
        )

    return (
        actor,
        reward_history,
        deviation_history,
        safety_history
    )

def train_a2c():

    env = LaneEnvironment()

    actor = Actor()

    critic = Critic()

    actor_optimizer = optim.Adam(
        actor.parameters(),
        lr=LEARNING_RATE
    )

    critic_optimizer = optim.Adam(
        critic.parameters(),
        lr=LEARNING_RATE
    )

    reward_history = []

    deviation_history = []

    safety_history = []

    for episode in range(
        EPISODES
    ):

        state = env.reset()

        log_probs = []

        values = []

        rewards = []
```

```
deviations = []

safety_values = []

for step in range(
    STEPS
):

    state_tensor = torch.FloatTensor(
        state
    ).unsqueeze(0)

    probabilities = actor(
        state_tensor
    )

    distribution = torch.distributions.Categorical(
        probabilities
    )

    action = distribution.sample()

    log_prob = distribution.log_prob(
        action
    )

    value = critic(
        state_tensor
    ).squeeze()

    (
        next_state,
        reward,
        done,
        deviation,
        safety
    ) = env.step(
        action.item()
    )

    log_probs.append(
        log_prob
    )

    values.append(
        value
    )

    rewards.append(
        reward
    )

    deviations.append(
```

```
        deviation
    )

    safety_values.append(
        safety
    )

    state = next_state

    if done:

        break

returns = calculate_returns(
    rewards
)

values = torch.stack(
    values
)

advantages = (
    returns
    -
    values.detach()
)

if len(advantages) > 1:

    advantages = (
        advantages
        -
        advantages.mean()
    ) / (
        advantages.std()
        +
        1e-8
    )

actor_loss = 0

for log_prob, advantage in zip(
    log_probs,
    advantages
):

    actor_loss += (
        -log_prob * advantage
    )

critic_loss = nn.functional.mse_loss(
    values,
    returns
)
```

```

    )

    actor_optimizer.zero_grad()

    actor_loss.backward()

    actor_optimizer.step()

    critic_optimizer.zero_grad()

    critic_loss.backward()

    critic_optimizer.step()

    reward_history.append(
        sum(rewards)
    )

    deviation_history.append(
        np.mean(deviations)
    )

    safety_history.append(
        np.mean(safety_values)
    )

    return (
        actor,
        reward_history,
        deviation_history,
        safety_history
    )

print("=====")
print("AUTONOMOUS LANE KEEPING")
print("PPO vs A2C")
print("=====")

print("\nTraining PPO...")

(
    ppo_actor,
    ppo_rewards,
    ppo_deviation,
    ppo_safety
) = train_ppo()

print(
    "PPO Training Completed."
)

```

```
print("\nTraining A2C...")

(
    a2c_actor,
    a2c_rewards,
    a2c_deviation,
    a2c_safety
) = train_a2c()

print(
    "A2C Training Completed."
)

window = 30

ppo_reward = np.mean(
    ppo_rewards[-window:]
)

a2c_reward = np.mean(
    a2c_rewards[-window:]
)

ppo_deviation_value = np.mean(
    ppo_deviation[-window:]
)

a2c_deviation_value = np.mean(
    a2c_deviation[-window:]
)

ppo_safety_value = np.mean(
    ppo_safety[-window:]
) * 100

a2c_safety_value = np.mean(
    a2c_safety[-window:]
) * 100

print("\n=====")
print("FINAL RESULTS")
print("=====")

print("\nPPO")

print(
    "Average Reward:",
    round(
```

```
        ppo_reward,
        3
    )
)

print(
    "Lane Deviation:",
    round(
        ppo_deviation_value,
        4
    )
)

print(
    "Safety:",
    round(
        ppo_safety_value,
        2
    ),
    "%"
)

print("\nA2C")

print(
    "Average Reward:",
    round(
        a2c_reward,
        3
    )
)

print(
    "Lane Deviation:",
    round(
        a2c_deviation_value,
        4
    )
)

print(
    "Safety:",
    round(
        a2c_safety_value,
        2
    ),
    "%"
)

print("\n=====")
print("COMPARISON")
```

```
print("=====")

print(
    "\nMetric".ljust(25),
    "PPO".ljust(15),
    "A2C"
)

print(
    "Average Reward".ljust(25),
    round(
        ppo_reward,
        3
    ),
    " " * 8,
    round(
        a2c_reward,
        3
    )
)

print(
    "Lane Deviation".ljust(25),
    round(
        ppo_deviation_value,
        4
    ),
    " " * 8,
    round(
        a2c_deviation_value,
        4
    )
)

print(
    "Safety (%)".ljust(25),
    round(
        ppo_safety_value,
        2
    ),
    " " * 8,
    round(
        a2c_safety_value,
        2
    )
)

if ppo_deviation_value < a2c_deviation_value:
```

```
    print(
        "\nPPO achieved lower lane deviation."
    )

else:

    print(
        "\nA2C achieved lower lane deviation."
    )

if ppo_safety_value > a2c_safety_value:

    print(
        "PPO achieved better safety."
    )

else:

    print(
        "A2C achieved better safety."
    )

if ppo_reward > a2c_reward:

    print(
        "PPO achieved higher reward."
    )

else:

    print(
        "A2C achieved higher reward."
    )

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    ppo_rewards,
    label="PPO"
)

plt.plot(
    a2c_rewards,
    label="A2C"
)

plt.xlabel(
```

```
        "Episode"
    )

    plt.ylabel(
        "Reward"
    )

    plt.title(
        "Reward Convergence"
    )

    plt.legend()

    plt.grid()

    plt.show()

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    ppo_deviation,
    label="PPO"
)

plt.plot(
    a2c_deviation,
    label="A2C"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Lane Deviation"
)

plt.title(
    "Lane Deviation Comparison"
)

plt.legend()

plt.grid()

plt.show()

plt.figure(
    figsize=(10, 5)
```

```

)

plt.plot(
    ppo_safety,
    label="PPO"
)

plt.plot(
    a2c_safety,
    label="A2C"
)

plt.xlabel(
    "Episode"
)

plt.ylabel(
    "Safety Rate"
)

plt.title(
    "Safety Comparison"
)

plt.legend()

plt.grid()

plt.show()

print("\n=====")
print("PROJECT COMPLETED")
print("=====")

```

```

=====
AUTONOMOUS LANE KEEPING
PPO vs A2C
=====

```

```

Training PPO...
PPO Training Completed.

```

```

Training A2C...
A2C Training Completed.

```

```

=====
FINAL RESULTS
=====

```

```

PPO
Average Reward: 234.524
Lane Deviation: 0.1428

```

Safety: 99.8 %

A2C
Average Reward: 253.059
Lane Deviation: 0.1315
Safety: 97.84 %

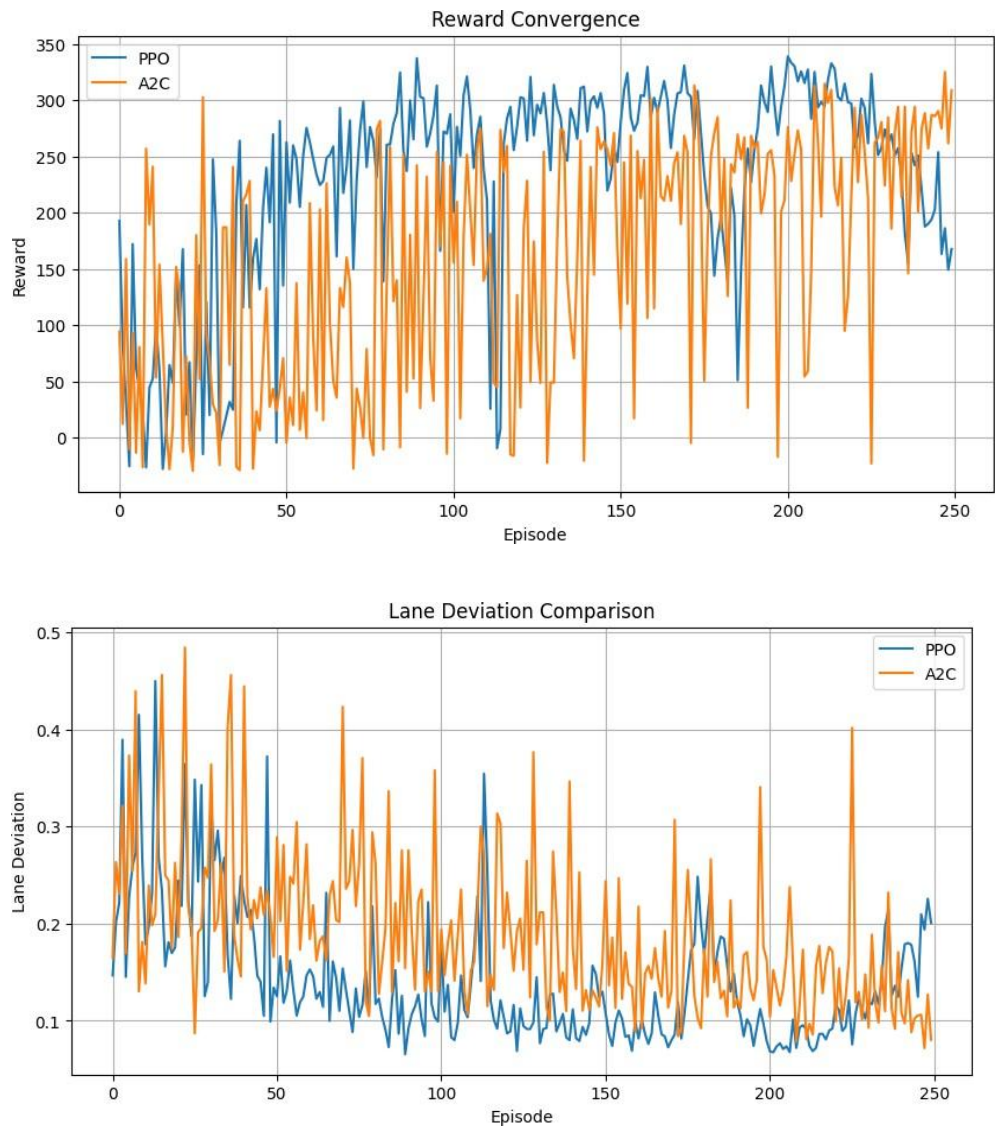
=====

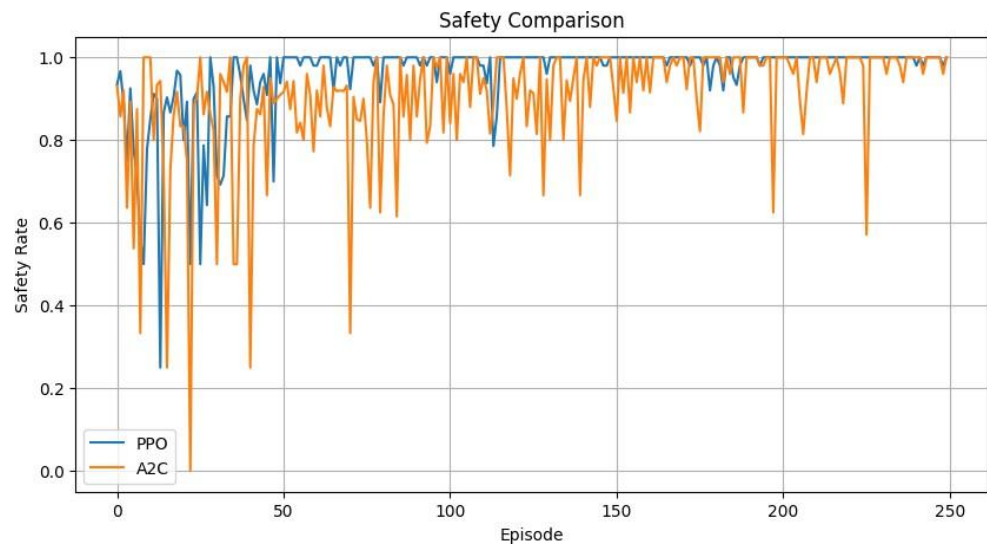
COMPARISON

=====

Metric	PPO	A2C
Average Reward	234.524	253.059
Lane Deviation	0.1428	0.1315
Safety (%)	99.8	97.84

A2C achieved lower lane deviation.
PPO achieved better safety.
A2C achieved higher reward.





=====
PROJECT COMPLETED
=====

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).